TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

Khwopa College of Engineering
Libali, Bhaktapur
Department of Computer and Electronics Engineering



A FINAL REPORT ON

CRIIQ: CRICKET INTELLIGENCE ENGINE

Submitted in partial fulfillment of the requirements for the degree
Bachelor of Computer Engineering

Submitted by

Anjana Silinchhe Shrestha	KCE080BCT004
Dipa Khanal	KCE080BCT007
Pradipta Joshi	KCE080BCT021
Sushma Shrestha	KCE080BCT043

Under the Supervision of
Er. Sushil Dyopala
Founder,
Tech Manthan

Khwopa College of Engineering
Libali, Bhaktapur
AUGUST, 2026

CERTIFICATE OF APPROVAL

This is to certify that this project report entitled **"CRIIQ: Cricket Intelligence Engine"**, submitted by Anjana Silinchhe Shrestha (KCE080BCT004), Dipa Khanal (KCE080BCT007), Pradipta Joshi (KCE080BCT021), Sushma Shrestha (KCE080BCT043) has been examined and accepted as the partial fulfillment of the requirements for the degree of Bachelor in Computer Engineering. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

.....
Er. Manjil Karki
External Examiner
Senior ML Engineer
coac GmbH Germany

.....
Er. Sushil Dyopala
Project Supervisor
Founder,
Tech Manthan

.....
Er. Dinesh Gothe
Head of Department
Department of Computer and Electronics Engineering
Khwopa College of Engineering

COPYRIGHT©

The author has agreed that the library, Khwopa College of Engineering, may make this report freely available for inspection. Moreover, the author has agreed that permission for the extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the project work recorded here or, in the absence of the supervisor, by the Head of the Department where the project report was done. It is understood that due recognition will be given to the author of this report and to the Department of Computer and Electronics Engineering, KhCE, in any use of the material of this project report. Copying, publication, or other use of this report for financial gain without the approval of the department and the author's written permission is prohibited. Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head of Department
Department of Computer and Electronics Engineering
Khwopa College of Engineering
Libali, Bhaktapur
Nepal

ACKNOWLEDGEMENT

We express our deep gratitude to Er. Dinesh Gothe, Head of the Department of Computer and Electronics Engineering, Khwopa College of Engineering (KhCE), for his regular support, cooperation, and coordination. We would also like to thank our supervisor Er. Sushil Dyopala for his guidance and valuable feedback throughout the project. We also thank the teaching and non-teaching staff of the Department of Computer and Electronics Engineering, KhCE, for their invaluable help and support.

Anjana Silinchhe Shrestha	KCE080BCT004
Dipa Khanal	KCE080BCT007
Pradipta Joshi	KCE080BCT021
Sushma Shrestha	KCE080BCT043

ABSTRACT

Cricket selection panels typically choose a playing eleven under time pressure, weighing form, head-to-head history, and conditions largely through experience rather than a systematic framework. Criiq is a cricket analytics and player-selection recommendation system for men's and women's T20 International cricket that scores every eligible player for a fixture and proposes a balanced, constraint-satisfying playing eleven, with plain-language reasoning attached to every pick. Since the proposal stage, the project has built a twelve-stage data-collection pipeline consolidating Cricsheet ball-by-ball data, ESPNcricinfo career metadata, and ICC rankings into a single SQLite database covering both genders' T20I history and several women's domestic leagues. This feeds a leakage-safe Win-Probability-Added (WPA) model, trained with gradient-boosted regression against a mean-predictor baseline on a temporal holdout, and a composite scoring function paired with a mixed-integer linear program that selects a rules-compliant Playing XI from a supplied twenty-player pool. The system also includes a ball-by-ball match simulator with genetic-algorithm optimizers for bowling rotation and batting order, and a FastAPI/React dashboard now fully wired to the trained recommendation engine and deployed to production, replacing the placeholder AI stub used during early frontend development. This report covers the work completed and the current architecture: the recommendation engine, SHAP-based explanations, and automated pool shortlisting from the full national squad are all implemented and deployed, with locally hosted LLM scouting reports and closing remaining coverage and evaluation gaps left as future work.

Keywords: *cricket analytics, T20I, player selection, win-probability-added, gradient boosting, constrained optimization, explainable recommendations*

Contents

Copyright i

Acknowledgement ii

Abstract iii

List of tables viii

List of figures ix

List of Symbols and Abbreviation xi

1 INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 2

1.3 Problem Statement 2

1.4 Goals and Objectives 2

1.5 Scope and Applications 3

2 LITERATURE REVIEW **4**

2.1 Literature Review 4

2.1.1 Puram, Roy, Srivastav, and Gurumurthy (2023) – Understanding the Effect of Contextual Factors and Decision Making on Team Performance in T20 Cricket: An Interpretable Machine Learning Approach 4

2.1.2 Lamsal and Kahle (2024) – In-Game Win Prediction Models for Cricket 4

2.1.3 Wickramasinghe (2022) – Applications of Machine Learning in Cricket: A Systematic Review 5

2.1.4 Hassan, Clough, Siddiqi, Ali, and Arshed (2024) – PlayerRank: Leveraging Learning-to-Rank AI for Player Positioning in Cricket 5

2.1.5 Chakraborty, Mondal, Bhattacharjee, Mallick, Santra, Maity, and Dey (2024) – Cricket Data Analytics: Forecasting T20 Match Winners Through Machine Learning 6

2.2 Related Theories 6

2.2.1 Win-Probability-Added as a Performance Label 6
2.2.2 Regression Model 7
2.2.3 The Composite Recommendation Score 7
2.2.4 XI Selection as Constrained Optimization 9
2.2.5 Evaluation Metrics 10
2.3 Review Matrix 11
2.4 Research Gap 11

3 METHODOLOGY **12**
3.1 System Block Diagram 12
3.2 Description of Working Flow of Proposed System 13
3.2.1 Data Collection 13
3.2.2 Data Preprocessing 13
3.2.3 Feature Engineering 14
3.2.4 Model Training 14
3.2.5 Evaluation 14
3.2.6 Recommendation 14
3.2.7 Serving 14
3.2.8 Monitoring and Maintenance 14
3.3 Model Architecture 15
3.3.1 WPA Regression Model Architecture 15
3.3.2 Composite Scoring and Constrained XI Selection Architecture 15
3.4 Hyperparameter Configuration 15
3.5 Model Evaluation Metrics 16
3.5.1 WPA Regression Metrics 16
3.5.2 Ranking and Selection Metrics 16
3.5.3 Win-Probability Metrics 16

4 SYSTEM DESIGN **17**
4.1 Requirement Specification 17
4.1.1 Functional Requirement 17

4.1.2 Non-Functional Requirement . . . 18
4.2 Feasibility Assessment . . . 18
4.2.1 Technical Feasibility . . . 18
4.2.2 Operational Feasibility . . . 18
4.2.3 Economic Feasibility . . . 19
4.2.4 Legal Feasibility . . . 19
4.3 Black Box Diagram . . . 19
4.4 Use Case Diagram . . . 20
4.5 Activity Diagram . . . 21
4.6 Class Diagram for System . . . 22
4.7 ER Diagram for Data . . . 23
4.8 Sequence Diagram . . . 24
4.9 Communication Diagram . . . 25
4.10 Data Flow Diagram . . . 26
4.11 Deployment Diagram . . . 27

5 RESULTS AND DISCUSSION 28

5.1 Experiment Setup . . . 28
5.1.1 Schedule (Gnatt Chart) . . . 28
5.1.2 Hardware and Software Requirements . . . 29
5.1.3 Data Collection Pipeline . . . 30
5.1.4 Player Identity Resolution and the Eligibility Registry . . . 31
5.1.5 Derived Analytics Layer . . . 31
5.2 Software Development Approach . . . 31
5.3 Model Deployment . . . 32
5.3.1 Backend Development . . . 32
5.3.2 Frontend Development . . . 32
5.3.3 Model Integration . . . 32
5.4 WPA Regression Model: Training and Analysis . . . 33
5.4.1 Win Probability and Win-Probability-Added Modelling . . . 33

5.4.2 Model Evaluation and Robust Comparison 34
5.5 Outputs and Results for WPA Regression Model 36
5.6 Recommendation Engine and Constrained XI Selection: Analysis and Results 38
5.6.1 Recommendation Engine and Constrained XI Selection . . . 38
5.6.2 Constrained XI Selection Results 38
5.6.3 Why MILP Remains Necessary 39
5.7 Graph Results 41
5.8 Match Simulation and Genetic-Algorithm Optimizer: Training and Evaluation 42
5.8.1 Match Simulator and Genetic-Algorithm Optimizer Design . 42
5.8.2 Genetic-Algorithm Optimizer Results 42
5.8.3 Evaluation Discussion 43
5.9 Explainability via SHAP-based Attribution 44
5.10 Testing 47

6 CONCLUSION **48**
6.1 Conclusion 48
6.2 Recommendation 48
6.3 Limitation and Future Work 49

REFERENCES **52**

A User Interface Screenshots **53**
A.1 Dashboard 53
A.2 Recommendation Explainability View 53
A.3 Match Simulator 54
A.4 Player Form 55

List of Tables

2.1 Components of the composite recommendation score 8

2.2 Review matrix of related works 11

5.1 Win-probability model (`EmpiricalWinProb`) performance, held out by match, not by ball 33

5.2 Win-probability model performance on 149 genuinely unseen forward-holdout matches 33

5.3 WPA regressor evaluation on a temporal holdout, shipped models . 34

List of Figures

2.1 Composition of the recommendation score 8

2.2 XI selection as a constrained optimisation problem 9

3.1 Block diagram of the Criiq system 12

3.2 Working flow of the Criiq system 13

4.1 Black Box Diagram 19

4.2 Use Case diagram of the System 20

4.3 Activity Diagram of the system 21

4.4 Class Diagram of the System 22

4.5 ER Diagram of the Data 23

4.6 Sequence Diagram 24

4.7 Communication Diagram 25

4.8 Data Flow Diagram 26

4.9 Deployment Diagram of the System 27

5.1 Gnatt Chart for the Criiq Project 29

5.2 Predicted vs. actual WPA per match, with binned-mean calibration curve, batting and bowling 36

5.3 Predicted vs. actual WPA, per-player average over players with $\geq$ 5 test matches ($n = 653$) 37

5.4 Residuals (predicted $-$ actual) on the temporal holdout, both genders and disciplines 37

5.5 Mean overlap with the real historical starting XI: AI-recommended vs. random-11 baseline 39

5.6 Bowling-rotation GA: best-known expected runs conceded by generation, three seeds 41

5.7 Batting-order GA: best-known expected score by generation, three seeds 41

5.8 Mean absolute SHAP value by feature, top 12 features, both genders and disciplines 45

5.9 SHAP beeswarm, female batting WPA (holdout sample, n = 300) . 46

5.10 SHAP beeswarm, female bowling WPA (holdout sample, n = 300) . 46

5.11 SHAP beeswarm, male batting WPA (holdout sample, n = 300) . . 47

5.12 SHAP beeswarm, male bowling WPA (holdout sample, n = 300) . . 47

A.1 Match Command Center dashboard 53

A.2 Recommendations page: "Why this recommendation" explain view with feature-impact attribution 54

A.3 Match simulator: XI selection, win probability, and ball-by-ball scorecard 54

A.4 Player Form scouting view 55

List of Symbols and Abbreviation

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command Line Interface
CSV	Comma-Separated Values
DB	Database
ESPN	Entertainment and Sports Programming Network
GA	Genetic Algorithm
H2H	Head-to-Head
HTTP	Hypertext Transfer Protocol
ICC	International Cricket Council
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MAE	Mean Absolute Error
MILP	Mixed-Integer Linear Program
ML	Machine Learning
NDCG	Normalized Discounted Cumulative Gain
ODI	One Day International
R^2	Coefficient of Determination
REST	Representational State Transfer
RSS	Really Simple Syndication
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
SQLite	Structured Query Language Lite (embedded database engine)
T20I	Twenty20 International
UI	User Interface
WPA	Win-Probability-Added
XI	Playing Eleven

CHAPTER 1

INTRODUCTION

1.1 Background

Cricket is one of the most data-rich team sports in the world. Structured ball-by-ball data is publicly available for international cricket back to 2001 for men's and 2003 for women's matches, through datasets such as Cricsheet [1], which now cover thousands of matches across the Test, One Day International (ODI), and T20 International (T20I) formats for both men's and women's programmes. Career statistics, retirement notices, playing styles, and rankings are similarly available from public sources such as ESPNcricinfo, Wikipedia, and the International Cricket Council (ICC). Criiq is built entirely on these publicly available player performance statistics and other publicly available cricket data sources, without relying on any proprietary or paid data feed.

Women's international cricket in particular has grown sharply in scale and institutional support over the past decade. The ICC Women's T20 World Cup prize pool rose from a total of USD 2.45 million in 2023 to a record USD 7.96 million in 2024 [2], which the ICC described as aligning with its vision for parity between the men's and women's game, even though the Men's T20 World Cup 2024 pool (USD 11.25 million) remained larger on account of ten additional teams and 32 more matches [3]. The ICC's Global Growth Strategy commits to giving women's events equal standing and recognition alongside the men's game, as part of a broader plan to build direct relationships with 300 million fans by 2032 [4]. Since April 2018, full T20I status has been extended to women's teams across all ICC member nations [5], so boards with little or no dedicated analytics infrastructure now field sides in the same international format as the highest-ranked full members.

Despite this abundance of public data, team selection still relies almost entirely on the accumulated experience and judgment of selectors and coaches. Choosing a playing eleven for a specific fixture means weighing recent form, head-to-head history against the specific opposition, ground and weather conditions, and the balance of batting, bowling, and wicket-keeping roles in the squad, all of which are decisions currently made without any systematic, reproducible framework.

1.2 Motivation

The motivation for this project comes from the gap between the volume of cricket data publicly available and the informal way most selection decisions are actually made. A selector who leaves out a player under public scrutiny cannot easily demonstrate whether that decision rested on genuine underperformance, an unfavourable match-up, or simple preference. A tool that scores every available player consistently, respects the structural constraints of a legal playing eleven, and explains its own reasoning in plain language gives selectors and coaches something they can act on and defend. Rather than replacing their judgment, it gives that judgment a transparent, data-grounded starting point.

A further motivation, carried through from the original project proposal, is that women's cricket specifically lacks published, deployable analytics tooling: every directly comparable study reviewed in Chapter 2 is built on men's competition data (IPL or unspecified men's T20I) [6–9], and the broader systematic review of the field reflects the same skew [10]. Building the data pipeline and modelling layer with the women's T20I programme as the default scope, and men's T20I as an equally supported second scope rather than an afterthought, closes a real and currently unaddressed gap.

1.3 Problem Statement

Team selection in T20I cricket is a constrained scoring and ranking problem: from a pool of eligible players, choose eleven who satisfy role and structural requirements (enough batting depth, enough bowling depth, a wicket-keeper, recognized openers) while maximizing expected contribution to winning the specific match at hand, given the opposition, the ground, and each player's current form and fitness. No existing free or openly documented tool performs this scoring and constrained selection end to end, explains its output in plain language, and treats data quality problems (player identity collisions, retired players, incomplete style records, blended-gender statistics) as first-class correctness concerns rather than afterthoughts. This project addresses that gap directly, using publicly available ball-by-ball and career data as its only inputs.

1.4 Goals and Objectives

The objective of this project is:

- To build and evaluate a data-driven player scoring and constrained team-selection system for men's and women's T20I cricket, trained on publicly available ball-by-ball and career data, that recommends a legal, balanced playing eleven for a given fixture with reasoning that a selector or coach without a data background can read and act on.

1.5 Scope and Applications

The project covers T20 International cricket for both genders, with women's T20I as the primary scope and men's T20I as an equally trained and evaluated second scope. Several women's domestic T20 leagues are ingested to supply a form signal for players who are quiet internationally but active domestically. One Day International and Test cricket, and men's domestic T20 leagues, are scaffolded in the codebase but not currently enabled. All data is sourced from openly licensed or publicly accessible repositories, and no proprietary or paid data source is used. The system recommends the best available eleven for a specific fixture from a supplied pool of twenty players. A separate, earlier stage that automatically shortlists that twenty-player pool from the full national pool for an entire tour or tournament has also been implemented, completing the two-stage selection cycle described in the original proposal, and the system is now deployed to production.

This report is organized as follows. Chapter 2 reviews the literature that informs the project's design choices and presents the related theoretical background: the scoring model, the constrained selection formulation, and the evaluation metrics used throughout. Chapter 3 describes the system's block diagram, working flow, model architecture, hyperparameter configuration, and model evaluation metrics. Chapter 4 presents the system design in detail, including requirement specification, feasibility, and the full set of UML-style diagrams describing the system's structure and behaviour. Chapter 5 presents the results and discussion obtained from every implemented module, covering the experiment setup, the software development approach, model deployment, and the outputs and evaluation of every trained model, discussed against real baselines. Chapter 6 concludes the report and states its recommendations, limitations, and future work. Screenshots of the working interface are collected in Appendix A.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Review

2.1.1 Puram, Roy, Srivastav, and Gurumurthy (2023) – Understanding the Effect of Contextual Factors and Decision Making on Team Performance in T20 Cricket: An Interpretable Machine Learning Approach

This paper investigates how contextual factors and in-game decisions affect team performance in T20 cricket, using an interpretable machine learning approach. It analyzes nine IPL seasons comprising 563 matches to measure the practical impact of match context on results.

The authors compare several algorithms, including gradient boosting, regression trees, bagging, random forest, and Bayesian Additive Regression Trees, to determine how much various contextual variables contribute to team performance. Feature importance techniques are then applied to rank these effects, producing an interpretable view of which decisions matter most.

This paper directly supports the use of gradient boosting and explainability methods in Criiq, but it studies overall team performance rather than generating a recommended playing eleven [6].

2.1.2 Lamsal and Kahle (2024) – In-Game Win Prediction Models for Cricket

This paper develops in-game win prediction models for cricket, focusing on estimating win probability as a T20 match unfolds ball by ball. It uses IPL data and evaluates model performance using the Brier score.

The authors build models that continuously update win probability throughout an innings based on the evolving match state, then assess prediction quality using the Brier score, a standard metric for probabilistic forecasts. This provides a rigorous, ball-by-ball evaluation of how accurately win probability can be estimated in real time.

This is the closest existing match to Criiq's win-probability component and validates the use of the Brier score as an evaluation metric, but it predicts match outcomes rather than individual player contribution or team selection [8].

2.1.3 Wickramasinghe (2022) – Applications of Machine Learning in Cricket: A Systematic Review

This paper presents a systematic review of two decades of machine learning research applied to cricket, covering studies published between 2001 and 2021. It follows the PRISMA methodology to ensure a structured and unbiased review process.

The review surveys the range of machine learning techniques applied across cricket analytics, including performance prediction, match outcome forecasting, and player evaluation, summarizing common approaches and trends over the two-decade period. It highlights which problems have received significant research attention and which remain comparatively unexplored.

This review maps the broader research landscape and helps justify why a combined scoring, prediction, selection, and explanation pipeline like Criiq remains a novel contribution, since no single existing system unifies all of these components [10].

2.1.4 Hassan, Clough, Siddiqi, Ali, and Arshed (2024) – PlayerRank: Leveraging Learning-to-Rank AI for Player Positioning in Cricket

This paper introduces PlayerRank, a system that uses learning-to-rank techniques to identify the best players for specific batting-order positions in T20 International cricket. It focuses specifically on optimizing player placement within the batting order.

The model is trained to rank players based on their suitability for a given batting-order slot, using historical T20I performance data rather than treating all players as interchangeable. This produces ranked recommendations for each position in the order rather than a single overall score per player.

This is the closest published work to Criiq's ranked player recommendations, but it focuses specifically on batting-order placement rather than selecting a full, rule-compliant playing eleven [9].

2.1.5 Chakraborty, Mondal, Bhattacharjee, Mallick, Santra, Maity, and Dey (2024) – Cricket Data Analytics: Forecasting T20 Match Winners Through Machine Learning

This paper compares multiple machine learning algorithms for forecasting the winner of T20 cricket matches using pre-match features. It evaluates logistic regression, support vector machines, random forest, decision tree, and XGBoost side by side.

Each algorithm is trained and tested on the same set of pre-match statistics, allowing a direct comparison of predictive accuracy across methods, with random forest achieving the highest accuracy at 84.06%. This provides a clear benchmark for how different algorithms perform on cricket match-outcome prediction.

This paper offers a useful accuracy benchmark for match-outcome models, but like other outcome-prediction papers, it does not extend to individual player scoring or team selection [7].

2.2 Related Theories

2.2.1 Win-Probability-Added as a Performance Label

Rather than training on raw batting or bowling statistics, the model-training module predicts **Win-Probability-Added (WPA)**: the sum, over every ball a player faced or bowled in a match, of the change in their team's win probability attributable to that ball. Given a per-ball win-probability function $W(s)$ estimated empirically from historical deliveries at game state s (runs, wickets, balls remaining, innings), a player's match WPA is

$$\mathrm{WPA}_p = \sum_{b \in B_p} \big(W(s_{b+1}) - W(s_b)\big) \cdot \sigma_p(b) \tag{2.1}$$

where B_p is the set of balls involving player p and $\sigma_p(b) \in \{-1, +1\}$ orients the sign so that a positive value always means the player's side is better off. Batting WPA and bowling WPA are trained and reported as two separate targets. This label is preferred over raw run/wicket counts because it is win-relevant by construction: a six struck in a dead rubber late in a lost cause contributes far less WPA than the same six struck during a tight run chase.

2.2.2 Regression Model

Both WPA targets are fitted with `HistGradientBoostingRegressor`, a histogram-based gradient boosting ensemble of the form

$$\hat{f}(x) = \sum_{m=1}^{M} \nu \cdot h_m(x) \tag{2.2}$$

where each h_m is a shallow regression tree fitted to the residual of the ensemble so far, ν is a learning-rate shrinkage factor, and M is the number of boosting rounds. This family was selected empirically: a robust comparison against Linear Regression, Ridge, Random Forest, XGBoost, and LightGBM on the same leakage-safe features and the same temporal holdout, repeated across three holdout fractions and five random seeds per stochastic model, found no model family reliably beating a simple mean-predictor baseline by a wide margin, and confirmed the gradient-boosted choice as the best available correlation-ranking performer for both targets (Chapter 5).

2.2.3 The Composite Recommendation Score

A player's final recommendation score, on a scale of 0 to 100, is not the raw model output. It is a documented, additive sum of parts, each independently bounded and each carrying the raw values that produced it:

$$\text{Score}(p) = \text{model}(p) + \text{matchup}(p) + \text{phase}(p) + \text{standing}(p) + \text{risk}(p) + \text{strike}(p) + \text{need}(p) \tag{2.3}$$

Figure 2.1 illustrates the composition, and Table 2.1 lists each component, its valid range, and what it measures.

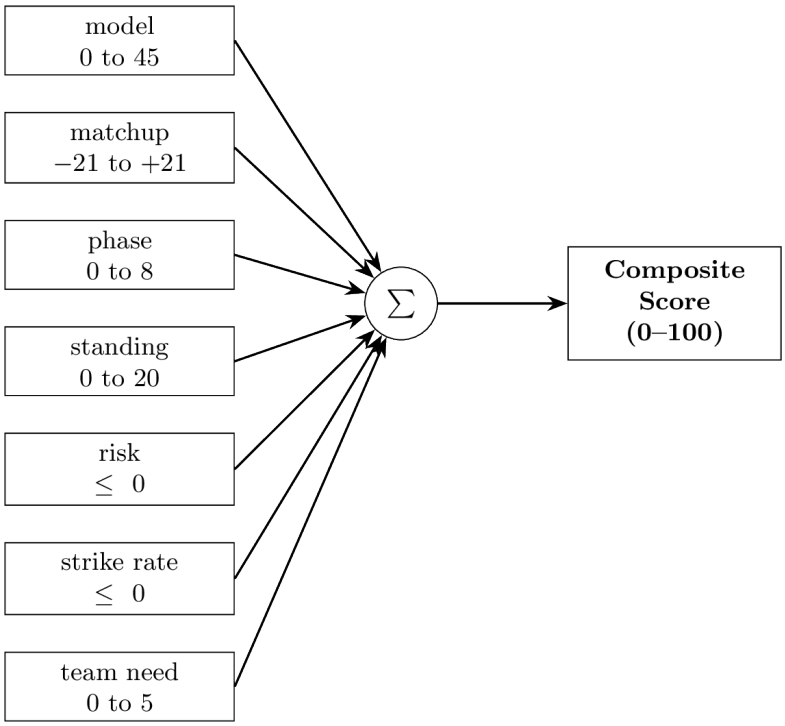


Figure 2.1: Composition of the recommendation score

Table 2.1: Components of the composite recommendation score

Component	**Range**	**What it measures**
model	0 to 45	Predicted batting + bowling WPA (or a sample-confidence-scaled shrinkage fallback below 10 career matches); the stronger discipline at full weight, the weaker at reduced weight.
matchup	−21 to +21	Batter-vs-opponent dismissal record, style-matchup edge, and bowler-vs-opponent record, each shrunk toward a prior.
phase	0 to 8	Strength in the player's best batting phase and best in-team bowling phase.
standing	0 to 20	Career importance score plus an ICC top-10 ranking bonus.
risk	≤ 0	Penalties only: age advisory, inconsistent selection history, inactive-but-recalled status.
strike	≤ 0	Soft penalty for below-par career strike rate, discounted by a game-awareness factor (experience, big-moment impact, chasing composure, consistency).
need	0 to 5	Bonus for a player whose strength lies in the pool's single weakest phase.

2.2.4 XI Selection as Constrained Optimization

Selecting the best eleven from a scored pool is not simply taking the top 11 scores: a legal side needs structure. Let $x_i \in \{0, 1\}$ indicate whether player i is selected. XI selection is formulated as a 0/1 mixed-integer linear program:

$$\max \quad \sum_{i=1}^{n} s_i x_i \tag{2.4}$$

$$\text{s.t.} \quad \sum_{i=1}^{n} x_i = 11 \tag{2.5}$$

$$\sum_{i \in \text{bat}} x_i \geq 5, \quad \sum_{i \in \text{bowl}} x_i \geq 5 \tag{2.6}$$

$$\sum_{i \in \text{keeper}} x_i \geq 1, \quad \sum_{i \in \text{opener}} x_i \geq 2 \tag{2.7}$$

$$x_i \in \{0, 1\} \quad \forall i \tag{2.8}$$

where s_i is player i's composite score and n is the size of the supplied pool, twenty players in the current implementation. Each minimum is capped at how many qualifying players actually exist in the pool, so a pool with no recorded keeper degrades the keeper constraint rather than becoming infeasible, and a greedy repair fallback (swap the best qualifying benched player for the worst non-qualifying selected player) is used if the solver itself fails. Figure 2.2 illustrates the pipeline from scored pool to selected eleven.

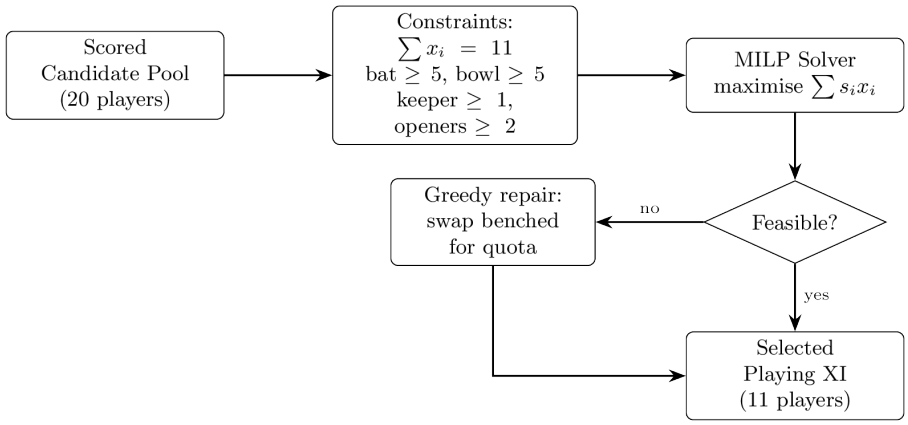


Figure 2.2: XI selection as a constrained optimisation problem

2.2.5 Evaluation Metrics

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.9}$$

Coefficient of determination (R^2):

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{2.10}$$

Pearson correlation between predicted and actual values, the primary ranking-quality metric because the engine ultimately consumes predictions as an ordering rather than as an absolute quantity:

$$r = \frac{\sum_i (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_i (y_i - \bar{y})^2} \cdot \sqrt{\sum_i (\hat{y}_i - \bar{\hat{y}})^2}} \tag{2.11}$$

Brier score, used to evaluate the ball-by-ball win-probability model against a match-level holdout:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - o_i)^2 \tag{2.12}$$

where p_i is the predicted win probability and $o_i \in \{0, 1\}$ is the actual outcome.

Log-loss:

$$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}\left[o_i \log p_i + (1 - o_i)\log(1 - p_i)\right] \tag{2.13}$$

Top-11 hit rate and **Spearman rank correlation** are used in the ranking backtest: the fraction of the actual top-11 performers (by real combined WPA in a completed match) that the model's predicted ordering also places in its own top 11, and the Spearman rank correlation between the two orderings within each held-out match. A discounted-gain ranking metric (NDCG) was deliberately not used, because its exponential gain formula misbehaves for a continuous, frequently negative quantity such as WPA.

2.3 Review Matrix

Table 2.2 summarises the reviewed work and maps each study to its relevance for this project.

Table 2.2: Review matrix of related works

Author / Year	**Approach / Focus**	**Limitation for this project**
Puram et al. (2023)	Interpretable ML (gradient boosting, BART) for contextual drivers of T20 team performance	Team-level scoring only; no selection framework
Wickramasinghe (2022)	Systematic review of ML applications in cricket (2001 to 2021)	Maps the research landscape; no unified pipeline proposed
Chakraborty et al. (2024)	Comparison of ML algorithms for pre-match T20 winner forecasting	Pre-match outcome only; no player scoring or selection
Lamsal & Kahle (2024)	Bayesian in-game win-probability model for IPL, evaluated via Brier score	Match outcome only; no player contribution or selection
Hassan et al. (2024)	PlayerRank: learning-to-rank for batting-order positioning	Batting-order ranking only; not full, rule-compliant XI selection
This project	Gradient-boosted WPA + composite score + MILP XI selection, both genders' T20I	Working, evaluated system; explainability and squad-level selection still in progress

2.4 Research Gap

The literature establishes that machine learning can predict match outcomes and evaluate player performance, and that explainability tools can make those predictions interpretable. No published, deployable system combines a leakage-safe win-probability-based performance model, a documented additive scoring function, constrained (rules-legal) eleven selection via integer programming, and a data pipeline that treats player-identity and gender-scoping correctness as an explicit engineering concern, applied to both men's and women's T20I with the women's programme as the default scope. Criiq addresses this gap. The extent to which it does so is documented and evaluated against real baselines in Chapter 5.

CHAPTER 3

METHODOLOGY

3.1 System Block Diagram

Figure 3.1 shows the overall system pipeline, from public data sources through to the served dashboard.

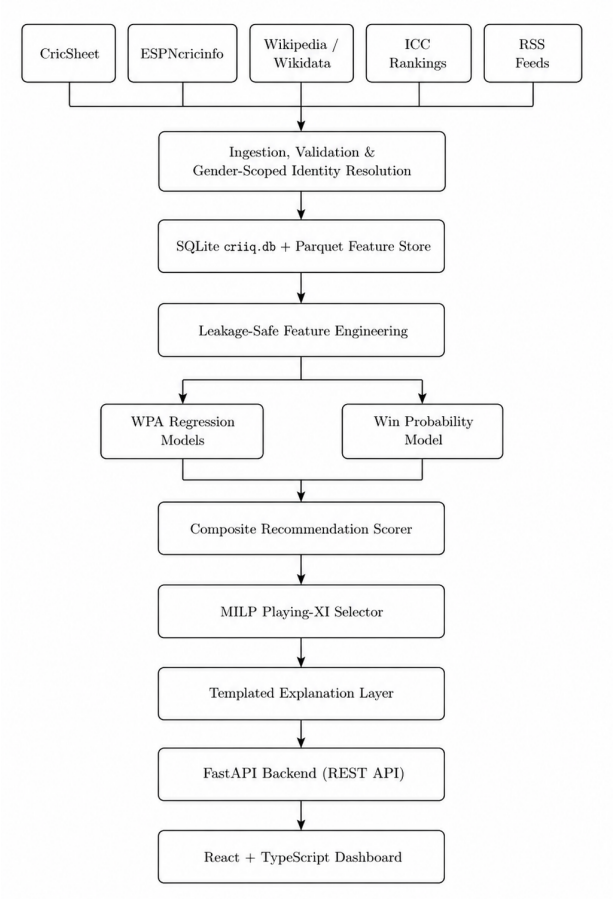


Figure 3.1: Block diagram of the Criiq system

3.2 Description of Working Flow of Proposed System

Figure 3.2 traces the working flow of the system end to end.

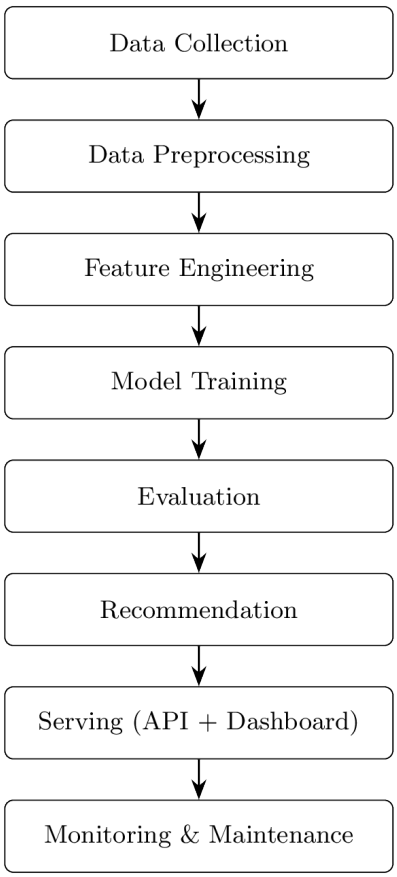


Figure 3.2: Working flow of the Criiq system

3.2.1 Data Collection

Ball-by-ball match data, career statistics, playing styles, ICC rankings, and retirement notices are collected from public sources on a scheduled, incremental basis.

3.2.2 Data Preprocessing

Ingested records are validated against an expected schema, player identifiers are resolved to a single canonical, gender-scoped player record, and venue names are normalised.

3.2.3 Feature Engineering

Point-in-time batting, bowling, matchup, phase, venue, and form features are computed for every player, strictly from information available before the match in question, so that no feature can leak information from the outcome being predicted.

3.2.4 Model Training

Two gradient-boosted regression models are fitted to predict batting and bowling Win-Probability-Added, using a chronological train/validation/test split.

3.2.5 Evaluation

Trained models are compared against a mean-predictor baseline and against alternative model families on a temporal holdout, using the metrics defined in Chapter 2.

3.2.6 Recommendation

For a given fixture, every eligible player in the supplied pool is scored with the composite function of Chapter 2, and a legal playing eleven is selected by solving the constrained optimisation problem.

3.2.7 Serving

Recommendations, player profiles, and win-probability estimates are exposed through a REST API and rendered in a web dashboard for coaches and selectors.

3.2.8 Monitoring and Maintenance

The pipeline is re-run on a scheduled basis to keep the database, rankings, and eligibility registry current, and the model-training and evaluation steps are re-run whenever the underlying features or training window change.

3.3 Model Architecture

3.3.1 WPA Regression Model Architecture

Both WPA targets are fitted with `HistGradientBoostingRegressor`, a histogram-based gradient boosting ensemble of the form

$$\hat{f}(x) = \sum_{m=1}^{M} \nu \cdot h_m(x) \tag{3.1}$$

where each h_m is a shallow regression tree fitted to the residual of the ensemble so far, ν is a learning-rate shrinkage factor, and M is the number of boosting rounds. This family was selected empirically over Linear Regression, Ridge, Random Forest, XGBoost, and LightGBM on the same leakage-safe features and the same temporal holdout, as detailed in Chapter 2 and evaluated in Chapter 5.

3.3.2 Composite Scoring and Constrained XI Selection Architecture

The model's raw output is one input among seven to the additive composite recommendation score defined in Chapter 2 (Equation 2.3). Selecting the eleven players to field from a scored pool is then posed as the 0/1 mixed-integer linear program of Chapter 2 (Equations 2.4–2.8), maximising total composite score subject to squad-size and role-minimum constraints, with a greedy repair fallback if the solver itself fails.

3.4 Hyperparameter Configuration

The gradient-boosting regressors are governed by the shrinkage factor ν and boosting-round count M of Equation 3.1; both were left at scikit-learn's `HistGradientBoostingRegressor` defaults rather than hand-tuned, since the robust model comparison (Chapter 5) showed no model family, this one included, reliably beating the mean-predictor baseline by a wide margin under tuning effort spent elsewhere. The robust comparison itself is run across three holdout fractions and five random seeds per stochastic model family, so that the reported ranking of model families is not an artefact of one lucky train/test split or one lucky seed. The constrained-selection stage has no learned hyperparameters: the role-minimum quotas (five batters, five bowlers, one keeper, two openers) and the pool size (twenty players) are fixed structural inputs, not tuned values.

3.5 Model Evaluation Metrics

Model quality is judged throughout using the metrics formally defined in Chapter 2. Every reported number is compared against its baseline rather than read in isolation, consistent with the project's evaluation discipline described in Chapter 5.

3.5.1 WPA Regression Metrics

Mean Absolute Error and R^2 against a mean-predictor baseline, and Pearson correlation as the primary ranking-relevant metric, are used to judge the two WPA regressors.

3.5.2 Ranking and Selection Metrics

Top-11 hit rate and Spearman rank correlation are used for the ranking backtest of the composite score and the constrained XI selector.

3.5.3 Win-Probability Metrics

Brier score and log-loss are used to evaluate the underlying per-ball win-probability model that the WPA label is derived from.

CHAPTER 4

SYSTEM DESIGN

4.1 Requirement Specification

4.1.1 Functional Requirement

- **FR1. Data Collection:** Collect ball-by-ball match data, career statistics, playing styles, rankings, and retirement notices from public sources for both genders' T20I cricket and selected women's domestic leagues.
- **FR2. Identity Resolution:** Resolve every player to a single canonical, gender-scoped identifier so that statistics are never blended across two different real people or across genders.
- **FR3. Eligibility Tracking:** Maintain a per-player, per-format activity status (active / inconsistent / inactive / retired / unknown) used to exclude ineligible players before scoring.
- **FR4. Feature Engineering:** Compute leakage-safe, point-in-time batting, bowling, matchup, phase, venue, and form features for every player.
- **FR5. Model Training:** Train and persist regression models that predict batting and bowling Win-Probability-Added, evaluated on a temporal hold-out against a mean-predictor baseline.
- **FR6. Recommendation:** Score every eligible player in a supplied pool for a given fixture and select a rules-legal playing eleven via constrained optimisation.
- **FR7. Explanation:** Attach a plain-language, component-traceable explanation to every recommended and non-recommended player.
- **FR8. Serving:** Expose recommendations, player profiles, saved teams, and win-probability estimates through a REST API and a web dashboard, with authenticated user accounts.

4.1.2 Non-Functional Requirement

- **NFR1. Reliability:** Pipeline stages are idempotent and safe to re-run. Retirement status changes require confirmation before being acted on by the eligibility gate.
- **NFR2. Performance:** Scoring a full candidate pool with explanations completes in well under a minute on standard hardware (a per-opponent roster cache reduced this from roughly 78 seconds to roughly 23 seconds during development).
- **NFR3. Transparency:** Every score is a documented sum of named components, each carrying its raw inputs. Explanations never state anything not present in those computed numbers.
- **NFR4. Correctness under real-world data:** Player-identity and gender-scoping bugs are treated as first-class defects, each found against live data, fixed at the root, and pinned with a regression test.
- **NFR5. Testability:** Both the data-collection and model-training modules run an offline test suite against small, in-memory fixture databases with no dependency on the live database.

4.2 Feasibility Assessment

4.2.1 Technical Feasibility

The project uses only openly available Python and JavaScript/TypeScript tooling (Chapter 5) and public data sources. Every core technique used, namely gradient-boosted regression, mixed-integer linear programming for constrained selection, and a FastAPI/React web stack, is mature, well documented, and already implemented and running in the codebase, confirming technical feasibility beyond the level of assumption typical at proposal stage.

4.2.2 Operational Feasibility

The system is designed to be operated by a coach or selector with no data background: every score is accompanied by a plain-language explanation, and the dashboard runs in a standard browser with no local installation. The backend

seeds a demo account automatically on first run, lowering the barrier to trying the system.

4.2.3 Economic Feasibility

All data sources used are free and either openly licensed or accessed within their public terms of use. Every library in the stack is free and open source, and no paid API or licensed software is required to run the system as it stands.

4.2.4 Legal Feasibility

The project sources data only from repositories that permit this kind of use, applies a polite, rate-limited scraping policy with an identifying user agent where scraping is used, and requires human confirmation before acting on an automated retirement alert, avoiding both legal and factual risk from acting on unverified news.

4.3 Black Box Diagram

At the highest level of abstraction, Criiq accepts a fixture context (the team, the opponent, an optional venue, and the pool of players to consider) and returns a ranked playing eleven, a win-probability estimate, a plain-language explanation for every player, and the player profiles and eligibility-registry alerts that support that recommendation. Figure 4.1 shows this input and output view without reference to internal structure.

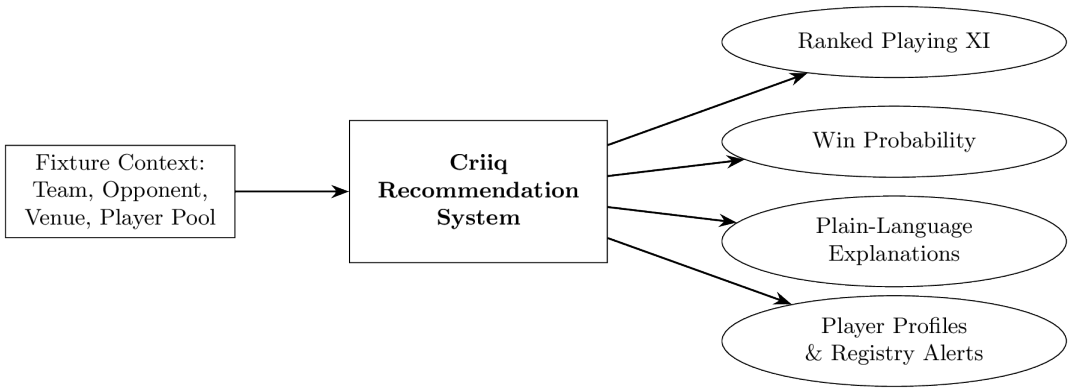


Figure 4.1: Black Box Diagram

4.4 Use Case Diagram

Three actors interact with the system: the coach or selector, who is the primary user and drives every player-facing use case; an administrator, who manages user accounts and can also request recommendations; and a background system scheduler, which periodically refreshes the data-collection pipeline so that the eligibility registry and player statistics stay current. Figure 4.2 enumerates the use cases each actor initiates.

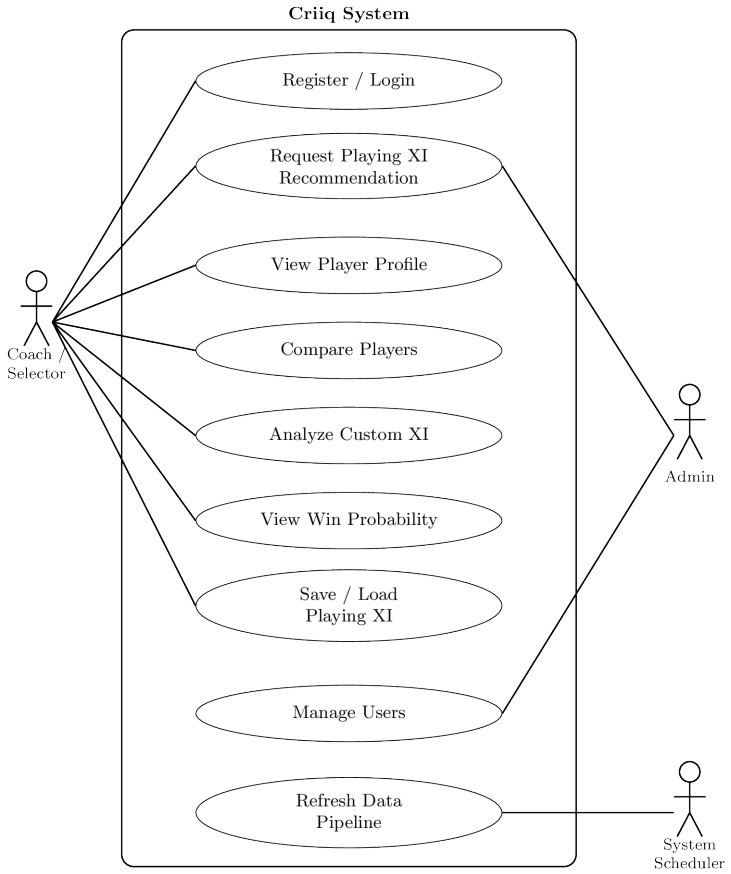


Figure 4.2: Use Case diagram of the System

4.5 Activity Diagram

Figure 4.3 traces the control flow of the system's central use case, requesting a playing-eleven recommendation, from the coach logging in and choosing a fixture through to the coach optionally saving the resulting team. The two decision points shown, namely whether the supplied pool has enough eligible players to proceed, and whether the constrained optimisation problem is solvable as posed, correspond directly to the pool-size guard and the MILP feasibility check described in Chapter 2.

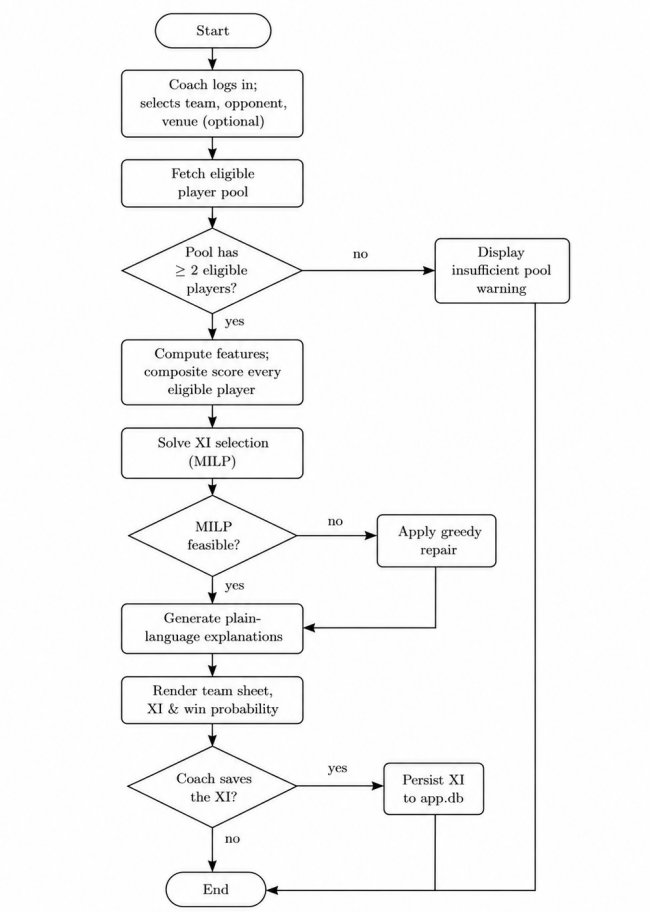


Figure 5.3: Activity Diagram of the system

Figure 4.3: Activity Diagram of the system

4.6 Class Diagram for System

Figure 4.4 shows the principal classes of the served application: authentication (User, Session), the fixture a recommendation is requested for (Fixture, Player), and the recommendation pipeline itself (RecommendationEngine, which delegates to a RecommendationScorer and an XISelector and produces one Explanation per scored player).

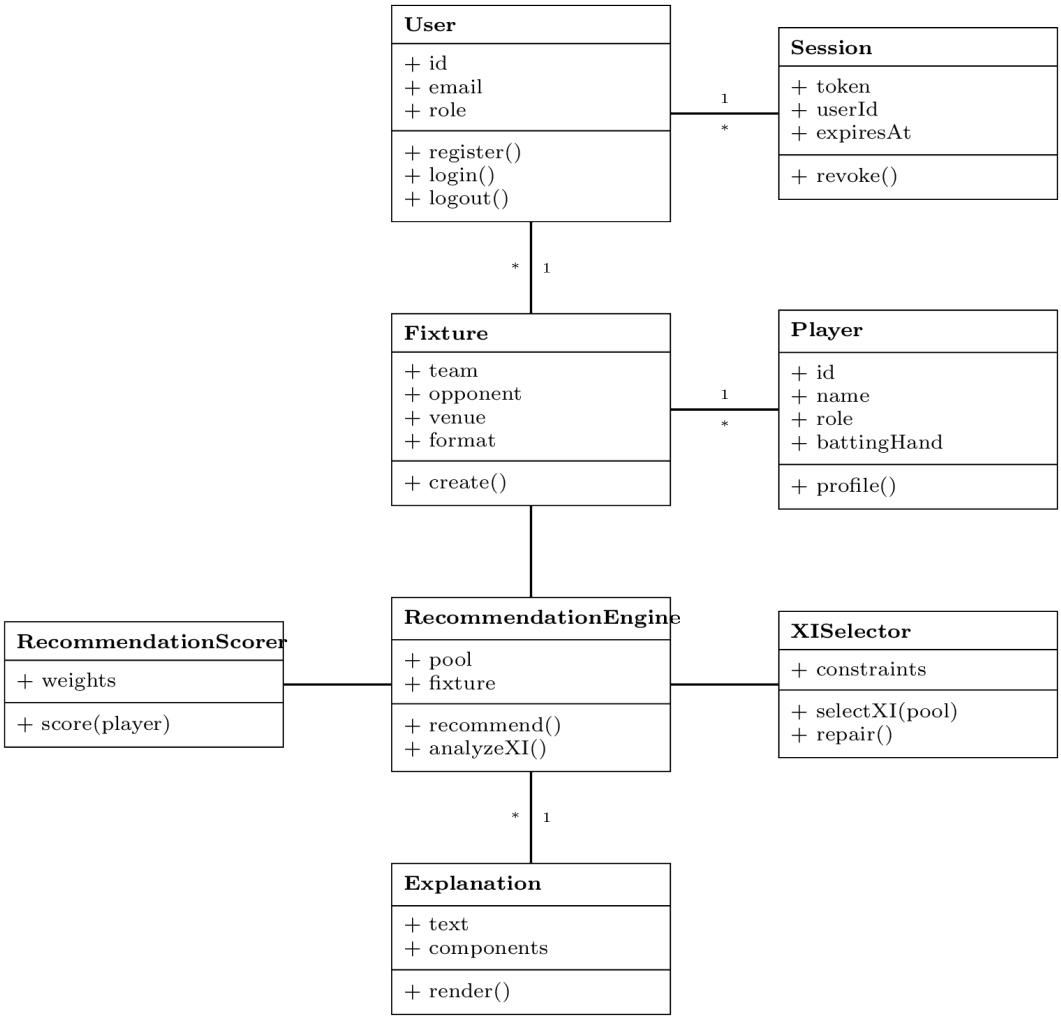


Figure 4.4: Class Diagram of the System

4.7 ER Diagram for Data

Figure 4.5 shows the principal entities persisted by the data-collection database: a canonical, gender-scoped Player record with its PlayerCareer, PlayerRegistry (eligibility) and PlayerCard (form/impact) records, per-match PlayerMatchWPA figures linked to Match and Delivery, and the DismissalMatchup table that records batter-versus-bowler history. This is a simplified view of the full schema described in Chapter 5.

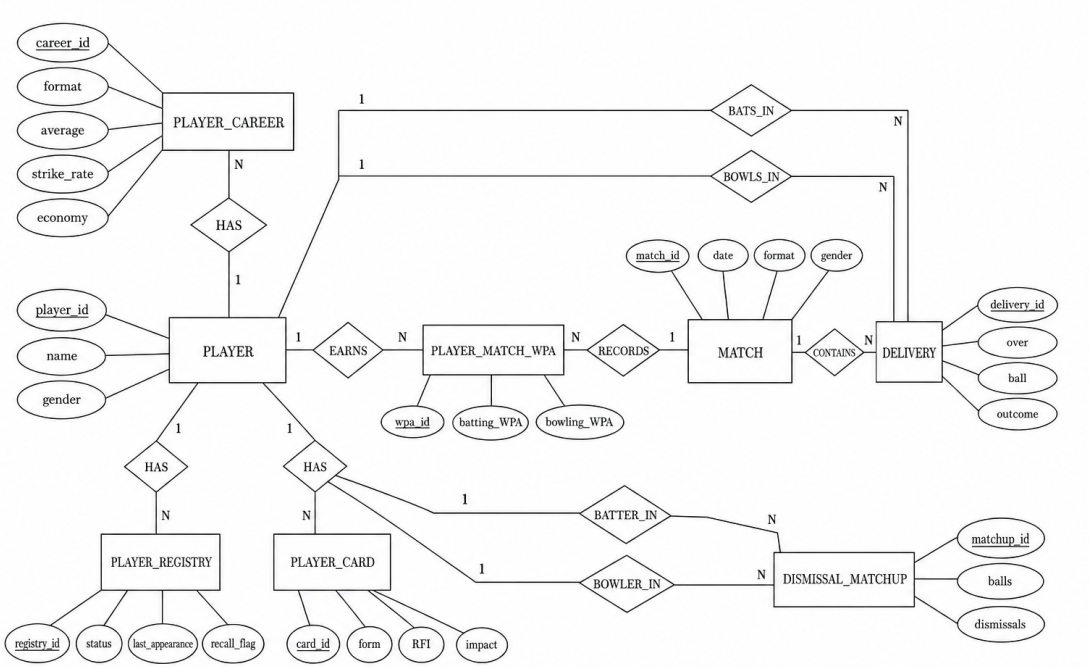


Figure 4.5: ER Diagram of the Data

4.8 Sequence Diagram

Figure 4.6 shows the time-ordered interaction between the coach, the frontend dashboard, the backend API, the recommendation engine, and the database for a single recommendation request, from the coach's initial fixture selection through to the rendered team sheet.

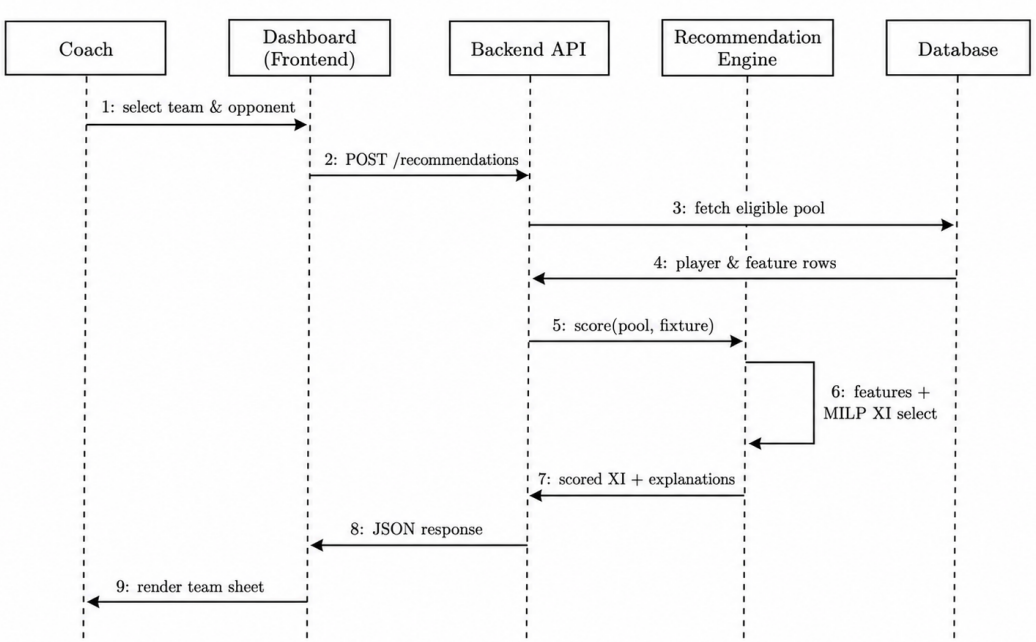


Figure 5.6: Sequence Diagram

Figure 4.6: Sequence Diagram

4.9 Communication Diagram

Figure 4.7 presents the same interaction as Figure 4.6, but organised around the objects involved and the numbered messages that pass between them rather than around elapsed time, emphasising the collaboration structure between the four participating objects.

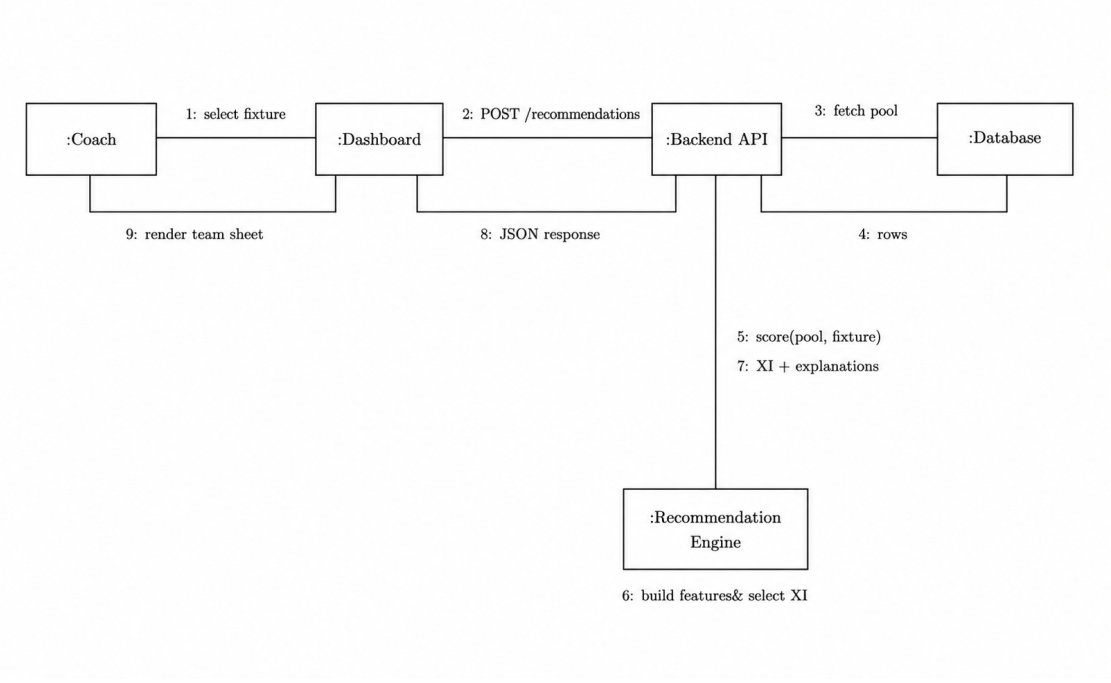


Figure 4.7: Communication Diagram

4.10 Data Flow Diagram

Figure 4.8 traces how data moves through the system as a whole: external sources feed an ingestion process that populates the primary database, which in turn feeds feature engineering, model training and scoring, and finally the selection and explanation process the coach interacts with directly. Data stores are drawn with the standard open-ended rectangle notation.

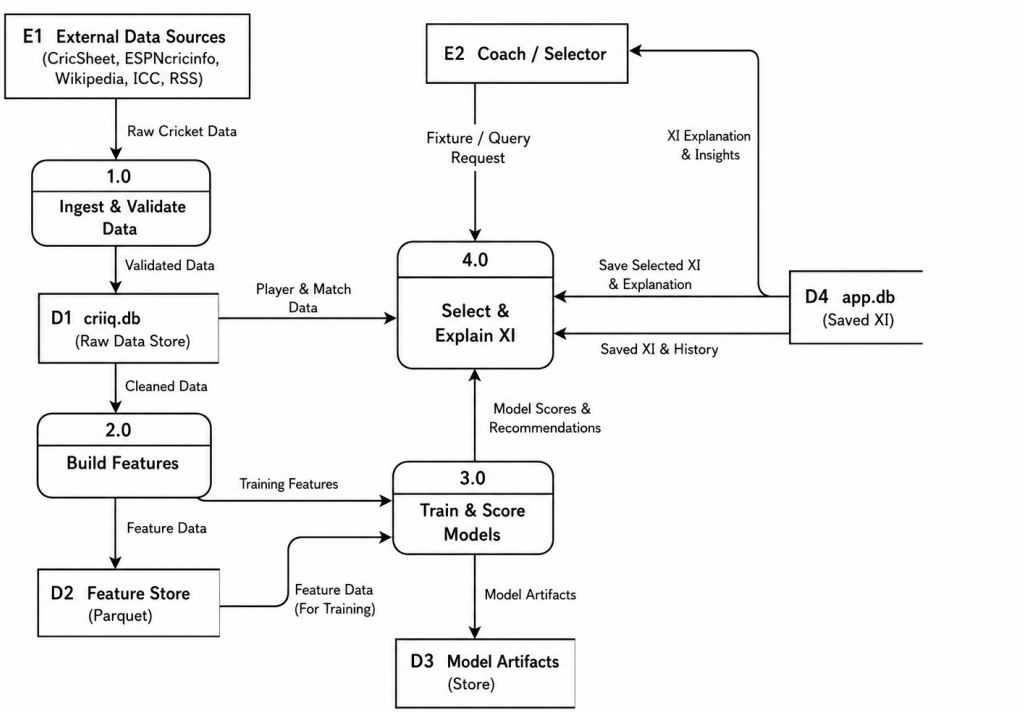


Figure 4.8: Data Flow Diagram

4.11 Deployment Diagram

Figure 4.9 shows the physical deployment structure: the React single-page application running in the coach's browser, communicating over HTTPS/REST with a FastAPI application server, which reads and writes the SQLite databases and trained model artifacts on the file system. A scheduled pipeline runner independently keeps the analytics database current.

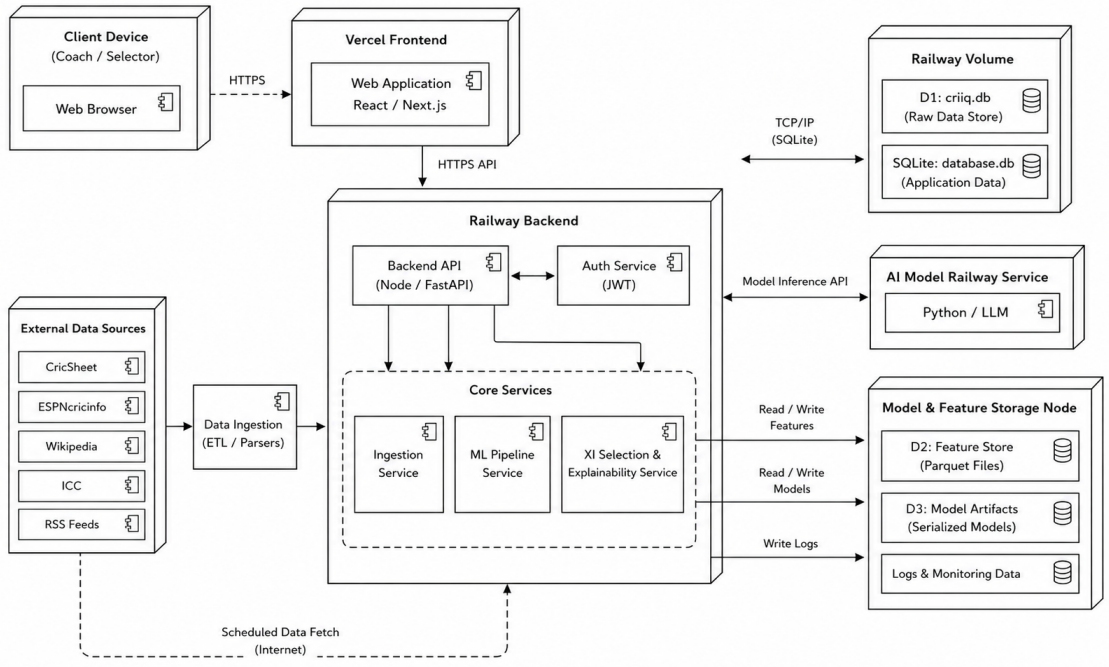


Figure 4.9: Deployment Diagram of the System

CHAPTER 5

RESULTS AND DISCUSSION

Development began on 19 June 2026. This chapter presents the experiment setup, the software development approach, model deployment, and the concrete results and outputs obtained from each implemented module: what was built, what it produces, and the measurable outcomes obtained from running it against real baselines, organised by module.

5.1 Experiment Setup

5.1.1 Schedule (Gnatt Chart)

Figure 5.1 shows progress against the eight-week, four-sprint schedule set out in the original proposal, at the completion of the project. The data-collection and model-training halves, the backend and frontend, the wiring of the trained recommendation engine into the served API (Section 5.3.3), SHAP-based explainability (Section 5.9), and the tour-squad selection stage were all delivered within the planned schedule, and the system is now fully deployed to production with no outstanding items.

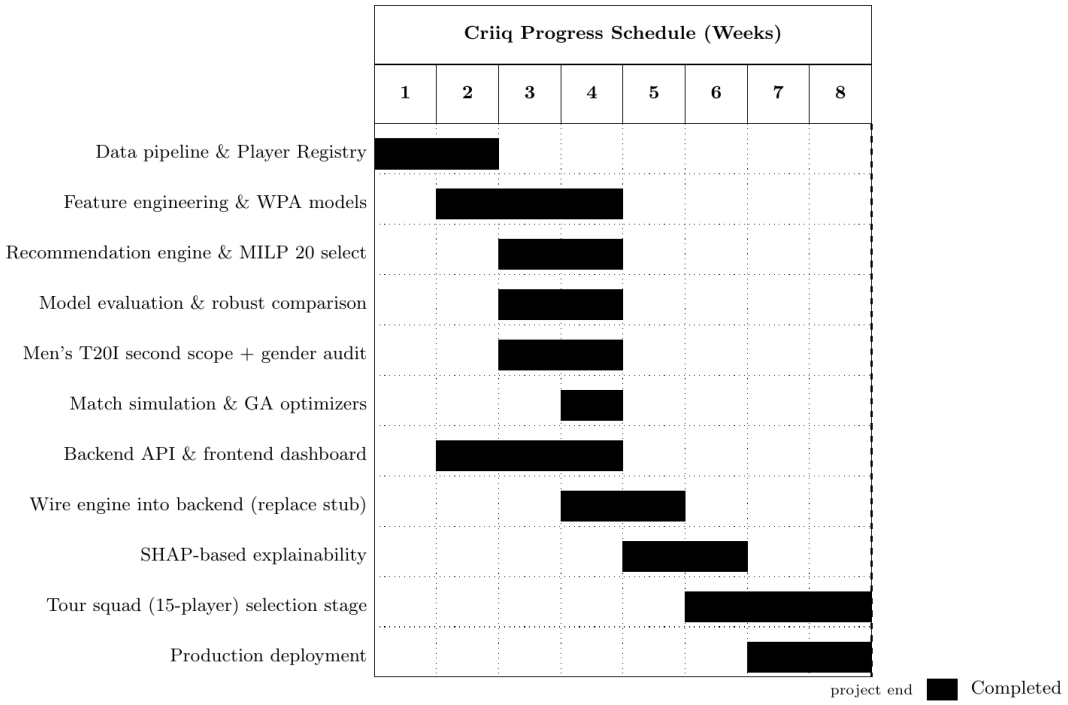


Figure 5.1: Gnatt Chart for the Criiq Project

5.1.2 Hardware and Software Requirements

The following software and languages are used in the project:

1. Python 3.11+ (data-collection), Python 3.13 (model-training)
2. pandas, NumPy for data manipulation and feature computation
3. scikit-learn (`HistGradientBoostingRegressor` and other model families) for WPA regression
4. SciPy (`optimize.milp`, `stats.spearmanr`) for constrained XI selection and rank evaluation
5. XGBoost and LightGBM as challenger models in the robust model comparison
6. joblib for trained-model persistence
7. SQLite for the primary analytics database, and Parquet (via PyArrow) for the ball-state and win-probability feature stores

8. requests, BeautifulSoup4, rapidfuzz, feedparser for data collection and enrichment
9. FastAPI with Uvicorn, PyJWT, and bcrypt for the backend REST API and authentication
10. React 18 with TypeScript, Vite, Tailwind CSS, React Router, Radix UI, and Framer Motion for the frontend dashboard
11. pytest for backend and model-evaluation unit tests
12. Jupyter notebooks for exploratory data analysis and model comparison
13. Git and GitHub for version control
14. Visual Studio Code as the primary IDE

The following hardware is sufficient to run every stage of the project:

1. A standard laptop or desktop CPU (quad-core or better); no GPU is required, since gradient-boosted regression on the current feature set trains in well under a minute
2. 8 GB RAM minimum, 16 GB recommended for comfortable notebook use
3. Approximately 2 to 5 GB of free disk space for the SQLite database, raw Cricsheet archives, and Parquet feature files
4. A standard internet connection for the initial and periodic data collection runs

5.1.3 Data Collection Pipeline

A twelve-stage, idempotent data-collection pipeline (data-collection package, 33 Python modules) has been built and is now in regular use. It downloads and ingests Cricsheet ball-by-ball data for both men's and women's T20 Internationals (over 5,400 international matches at the time of writing, plus several women's domestic T20 leagues: WBBL, WPL, The Hundred, Women's T20 Blast, Women's T20 Challenge, Women's CSL, and T20 Blaze), enriches it with career statistics, playing styles, and retirement status scraped from ESPNcricinfo and Wikipedia/Wikidata, and also ingests ICC rankings and RSS retirement feeds. Everything is written to a single self-describing SQLite database (`criiq.db`) plus a small set of Parquet feature files, and every stage is safe to re-run, with only new data processed on each run.

5.1.4 Player Identity Resolution and the Eligibility Registry

Player identity is resolved deterministically through Cricsheet's own people register, which carries the ESPNcricinfo player id, rather than by fuzzy name matching. During development this surfaced a genuine upstream data problem: Cricsheet's own per-match registry reuses the same identifier for two different real people of different genders in 32 confirmed cases. This was fixed at the schema level (identity is now resolved by (identifier, gender), not identifier alone), a change confirmed to remove 2,842 orphaned, blended rows across every derived table on rebuild. A `player_registry` table tracks, per player and per format, an activity status (active / inconsistent / inactive / unknown / retired), the most recent international appearance, a recall flag, and an age advisory. This is the working equivalent of the eligibility gate described in the original proposal, currently keyed on activity status rather than a fully separate retirement flag.

5.1.5 Derived Analytics Layer

On top of the raw ball-by-ball records, the pipeline builds: career batting and bowling summaries per player and format; dashboard-ready player cards (recent form, a Recent Form Index, team impact); archetype and phase-split profiles (powerplay/middle/death, vs-pace/vs-spin, vs-left/right-handed); full head-to-head and dismissal-matchup tables (batter-vs-bowler, minimum six balls, both bogey matchups and favourite victims); per-venue performance; bowling containment metrics (dot-ball percentage, boundary percentage, bowling strike rate); and a finalized batter/all-rounder/bowler role with a wicket-keeper flag, covering both genders across dozens of national teams.

5.2 Software Development Approach

The project followed an Agile development approach organised into four two-week sprints, as set out in the original proposal: Sprint 1 (data pipeline and Player Eligibility Registry), Sprint 2 (feature engineering and T20I models), Sprint 3 (recommendation engine, evaluation, simulation and optimisation tooling, and the web application), and Sprint 4 (integration, explainability, and deployment). Development has proceeded through all four sprints, with the data-collection and model-training halves considerably more mature than originally scoped, and the integration of the trained recommendation engine into the served application, SHAP-based explainability, and the tour-squad shortlisting extension all delivered within

Sprint 4 as planned. The system is now deployed to production with no tasks remaining. Within each sprint the team follows the usual cycle of planning, design, development, testing, and review, with commits scoped to a single, well-described unit of work so that regressions can be attributed and reverted precisely.

Agile suits this project because the feature weights, penalty magnitudes, and even the choice of model family are determined empirically from real data during development, and those results directly shape the next iteration's scope. A plan fixed in advance of seeing the data's actual predictive ceiling simply would not survive contact with it.

5.3 Model Deployment

5.3.1 Backend Development

A FastAPI backend exposes authentication (JWT-based login/registration), player listing and detail endpoints, match and fixture endpoints, a dashboard summary and alerts feed, saved playing-XI storage, and recommendation/win-probability endpoints, backed by the read-only analytics database plus two small read-write databases of its own (application data and authentication data, kept separate so that resetting one can never wipe user accounts).

5.3.2 Frontend Development

A React 18 and TypeScript frontend, built around a broadcast-style "Match Command Center" aesthetic with Tailwind CSS, React Router, and Framer Motion, provides a coach-facing application (dashboard, player profiles and comparison, fixtures, playing-XI selection) and a separate admin console, now connected end to end to the backend's authentication and data endpoints.

5.3.3 Model Integration

The AI-facing endpoints, previously served by a documented placeholder module (`ai_stub.py`), now serve recommendations and win-probability estimates from the trained `criiq_models` engine. The backend's recommendation and win-probability endpoints were originally served by this documented placeholder module rather than the trained `criiq_models` engine. Both modules were deliberately built to the same function signatures, so the placeholder has now been swapped out for

the trained engine as a drop-in replacement, with the model-training package and its artifacts packaged alongside the backend.

5.4 WPA Regression Model: Training and Analysis

5.4.1 Win Probability and Win-Probability-Added Modelling

An empirical, per-ball win-probability model (`EmpiricalWinProb`) is fitted separately for each combination of format and gender (previously a single shared model, corrected during a later gender-isolation audit). Table 5.1 reports its evaluation on a match-level holdout – split by match, never by ball, so no delivery from a held-out match is seen during training – against a constant-prior baseline.

Table 5.1: Win-probability model (`EmpiricalWinProb`) performance, held out by match, not by ball

Gender	Test balls	Brier	Brier baseline	Log-loss	Log-loss baseline
Female	93,714	0.1557	0.2447	0.4775	0.6825
Male	145,372	0.1727	0.2500	0.5156	0.6932

Lower is better for both metrics. The model meaningfully beats the constant-prior baseline in both genders, confirming that it is genuinely reading match state rather than reproducing the base win rate.

This is corroborated by a second, independent test: a built-in forward-holdout of 149 real matches (16 June to 13 July 2026, including the entire ICC Women's T20 World Cup and a men's India tour of England series) that the currently-shipped models have never seen in any form. Table 5.2 scores the same, unmodified win-probability model on this genuinely unseen set, alongside ball-level and start-of-chase match accuracy against a naive baseline.

Table 5.2: Win-probability model performance on 149 genuinely unseen forward-holdout matches

Gender	Test balls / matches	Brier	Brier baseline	Ball-level accuracy	Baseline accuracy	Start-of-chase match accuracy
Female	12,332 / 50	0.214	0.248	65.7%	54.6%	58.0%
Male	15,584 / 68	0.145	0.249	79.1%	52.8%	82.4%

The male scope actually outperforms its internal-split numbers here (Brier 0.145 vs. 0.173), while the female scope is comparatively weaker on this specific slice (58.0% start-of-chase accuracy, against a stronger internal-split figure) – plausibly because World Cup knockout matches are closer and higher-variance than the average historical match, and 50 matches is a small sample. Both genders still clearly beat their own baseline on every column, on data the model has never seen in any form.

This win-probability model is the basis for Win-Probability-Added (WPA): the per-match, per-player change in win probability attributable to their contributions with bat and ball, computed separately for batting and bowling and used as the training label for the recommendation engine's learned component.

5.4.2 Model Evaluation and Robust Comparison

Both WPA regressors are evaluated on a strict temporal holdout (never a random split, since rows within a match are correlated and a random split would let the model train on the future) against a mean-predictor baseline. Table 5.3 reports the full evaluation suite computed on the shipped models, covering Mean Absolute Error against the baseline, R^2, Pearson correlation, top-quartile hit rate, and sign-agreement accuracy, for both batting and bowling WPA across both genders. The low but positive R^2 values are expected rather than a bug: single-match WPA is a high-variance target dominated by individual big innings, so R^2 stays low even where the model extracts genuine signal, which is why Pearson correlation and top-quartile hit rate – the more informative columns for what is fundamentally a ranking task – are read alongside it rather than in isolation.

Table 5.3: WPA regressor evaluation on a temporal holdout, shipped models

Model	**MAE (baseline)**	R^2	**Pearson** r	**Top-quartile hit rate**	**Sign agreement**
Female batting WPA	0.0947 (0.0961)	0.0136	0.1254	35.3%	52.6%
Female bowling WPA	0.0772 (0.0785)	0.0063	0.0798	27.5%	55.5%
Male batting WPA	0.0887 (0.0896)	0.0074	0.0914	30.8%	51.8%
Male bowling WPA	0.0697 (0.0709)	0.0054	0.0752	25.3%	54.4%

A separate, more rigorous comparison repeats this across three holdout fractions and five random seeds per stochastic model for Linear Regression, Ridge, Random Forest, HistGradientBoosting, XGBoost, and LightGBM, confirming that the production model choice is not an artefact of one lucky split. Men's T20I was subsequently integrated as a fully trained and independently verified second scope, with its own registry-status verification across nine national teams rather than being

merely ingested. The constrained-selection stage and the two genetic-algorithm optimizers were evaluated with the same discipline; Section 5.6.3 onward present those results in full, with figures.

5.5 Outputs and Results for WPA Regression Model

A raw predicted-vs-actual scatter is a poor way to see the model's signal, since single-match WPA is dominated by match-to-match noise and predicted values cluster in a narrow band near zero regardless of whether the model is extracting real signal. Figure 5.2 addresses this by overlaying a decile-binned mean-prediction curve on top of the raw per-match scatter for the batting and bowling regressors, against the dashed perfect-calibration line ($r = 0.110$ for batting, $r = 0.056$ for bowling on individual matches). Figure 5.3 then aggregates the same predictions and actuals to a per-player average, restricted to players with at least five test-set appearances ($n = 653$ players), which averages out per-match noise and reveals a substantially stronger relationship ($r = 0.309$ for batting, $r = 0.198$ for bowling): the ranking signal is real, but only becomes visually clear once match-level noise is averaged away. Figure 5.4 shows the corresponding residuals for all four gender/discipline regressors, which are centred close to zero and roughly symmetric, with no visible bias toward over- or under-prediction for either gender or discipline.

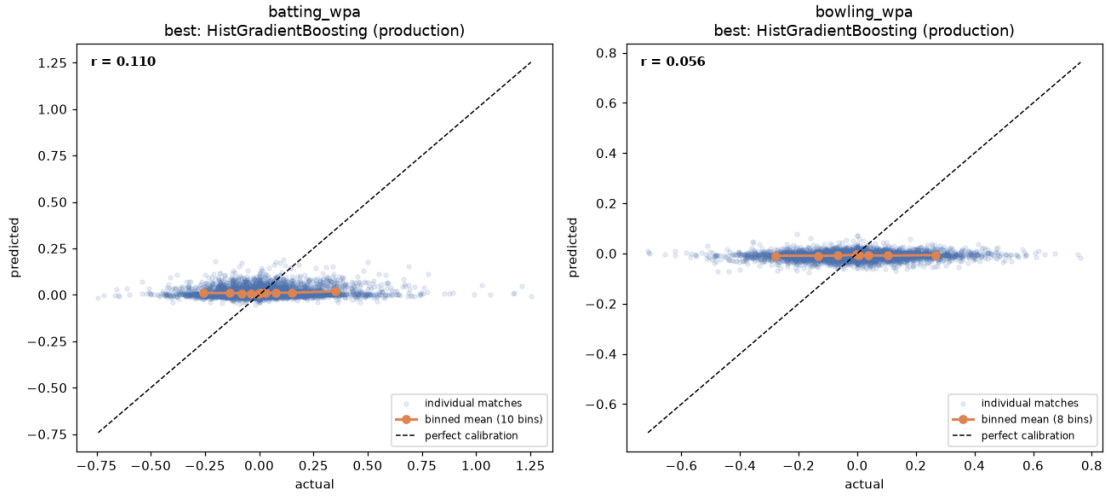


Figure 5.2: Predicted vs. actual WPA per match, with binned-mean calibration curve, batting and bowling

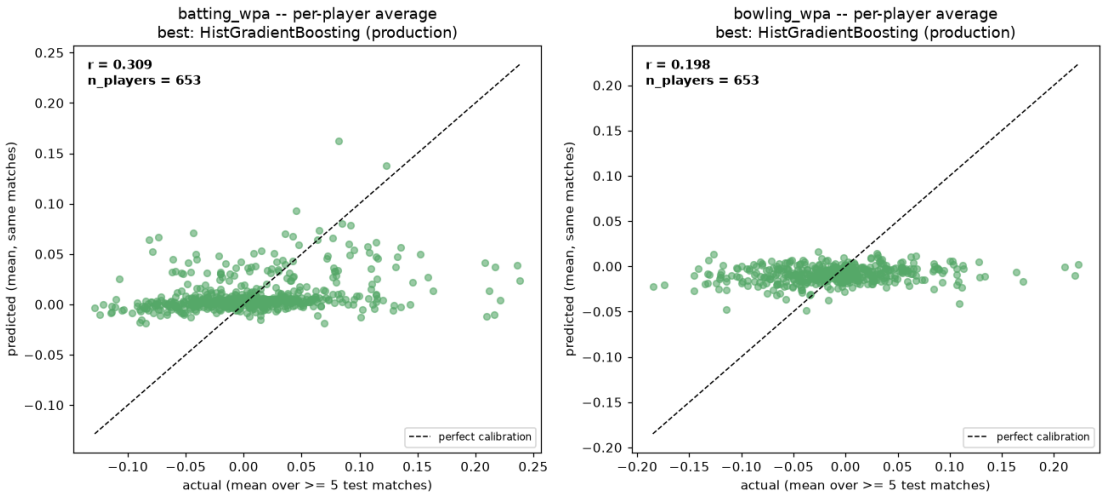


Figure 5.3: Predicted vs. actual WPA, per-player average over players with ≥ 5 test matches ($n = 653$)

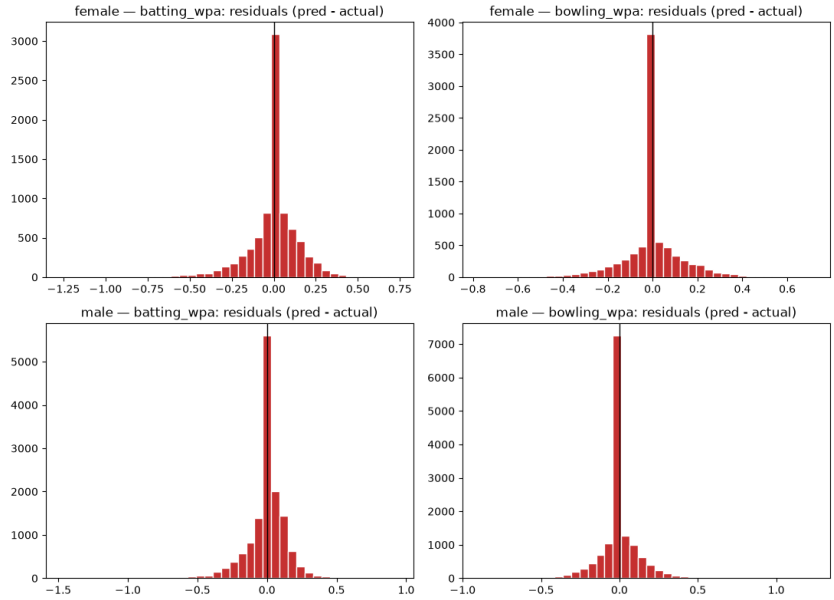


Figure 5.4: Residuals (predicted − actual) on the temporal holdout, both genders and disciplines

Read together with Table 5.3, the compressed prediction range explains why R^2 stays low (0.005–0.014) even though every model beats its mean-predictor baseline on MAE and shows a positive Pearson correlation: R^2 penalises exactly the variance the model deliberately does not chase, since single-innings WPA is dominated by rare, large individual performances that no pre-match feature set can foresee. The metrics that matter for a selection tool – Pearson correlation, top-quartile hit rate, and sign-agreement accuracy – are consistently positive and above chance

across all four models, which is the more relevant reading for a ranking task.

5.6 Recommendation Engine and Constrained XI Selection: Analysis and Results

5.6.1 Recommendation Engine and Constrained XI Selection

The model-training package (`criiq_models`) reads the data-collection database read-only and implements the full recommendation pipeline described in Chapter 2: leakage-safe feature engineering (with unit tests proving strict pre-match provenance), two gradient-boosted WPA regressors, and the seven-component additive composite score. The engine scores a supplied pool of twenty players for a given fixture and selects the Playing XI of eleven from that pool as a 0/1 mixed-integer linear program solved with SciPy, including a graceful constraint-degradation path and a greedy repair fallback. Built on top of this are counter-optimisation against a named opposition XI, a head-to-head matrix between a recommended side and a named opposition, lopsided-match detection with an automatically generated rotation XI, team-need synergy scoring for the twenty-player pool's weakest phase, and an analyze-xi command that scores and checks a coach's own hand-picked eleven against the same constraints the automatic selector is held to. All of this is reachable through a command-line interface with a human-readable team-sheet renderer in addition to raw JSON output.

5.6.2 Constrained XI Selection Results

Figure 5.5 visualises the real-world overlap result already summarised numerically above: across 18 genuinely unseen forward-holdout team-selections, the AI-recommended XI matched a mean of 7.50 of the 11 real historical starting players, against 5.26 of 11 for a random-11-from-pool baseline.

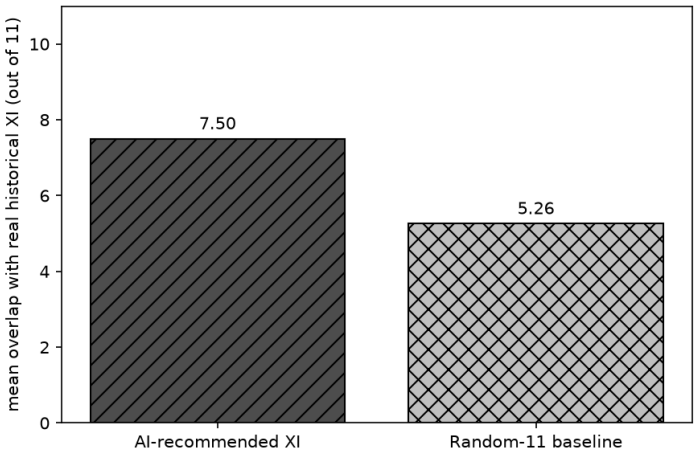


Figure 5.5: Mean overlap with the real historical starting XI: AI-recommended vs. random-11 baseline

The 100% constraint-compliance result (49 sampled pools) and the modest 1.8-point score cost of enforcing legality, against a naive unconstrained top-11 that would itself have broken a real constraint in 36.7% of the same pools, together support the design decision in Chapter 2 to solve selection as a constrained optimisation problem rather than a plain top-k ranking: the constraints are not free, but they are cheap, and skipping them fails often enough to matter. The sample so far is drawn only from well-covered women's T20I teams; extending this check to men's T20I and to smaller, less-covered squads is noted as a limitation and as future work in Chapter 6.

5.6.3 Why MILP Remains Necessary

The robust comparison of Section 5.4.2 extends to six regression families – Linear Regression, Ridge, Random Forest, HistGradientBoosting, XGBoost, and LightGBM – as candidates for the WPA component of the composite score. However well tuned, each of these solves the same kind of problem: given one player's features, output one number. None has any notion of the other players in the pool, so none can guarantee that a set of individually good scores adds up to a legal side. This is not hypothetical: taking the eleven highest composite scores directly – the naive top-11 baseline evaluated above – produced an illegal side (short of a role minimum, or missing a keeper) in 36.7% of the 49 sampled pools.

MILP is the stage that closes this gap. It takes the composite scores as fixed input and solves a genuinely different, combinatorial problem: choose the 0/1 combination of eleven players that maximises the sum of those scores subject to

the role and quota constraints of Chapter 2, holding simultaneously across all eleven players at once. Because SciPy's solver finds a certified exact optimum, the 100% constraint-compliance result comes with a guarantee no regression model can offer: there is provably no other legal combination with a higher total score. This guarantee is obtained cheaply – a 1.8-point average cost of legality against the illegal top-11 baseline – while still gaining 58.1 points over a random legal eleven, confirming the optimizer is doing substantial, non-trivial work.

The two stages are complementary rather than competing: regression only changes the numbers that reach the MILP's objective function, while MILP only chooses which eleven role-constrained players to take from those numbers. The genetic-algorithm optimizers discussed next sit downstream of both, taking an already-selected XI as given and solving a third, distinct problem: ordering and scheduling within that fixed set of eleven.

5.7 Graph Results

Figures 5.6 and 5.7 show fitness convergence across three random seeds for the bowling-rotation and batting-order genetic algorithms respectively, on one real India-vs-Australia fixture built from the system's own recommended XIs.

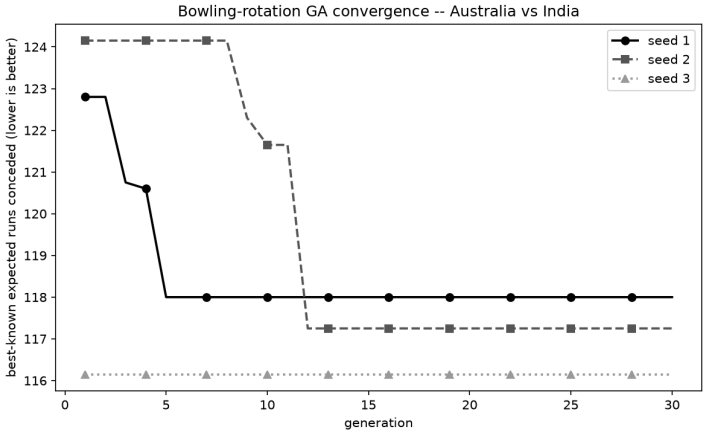


Figure 5.6: Bowling-rotation GA: best-known expected runs conceded by generation, three seeds

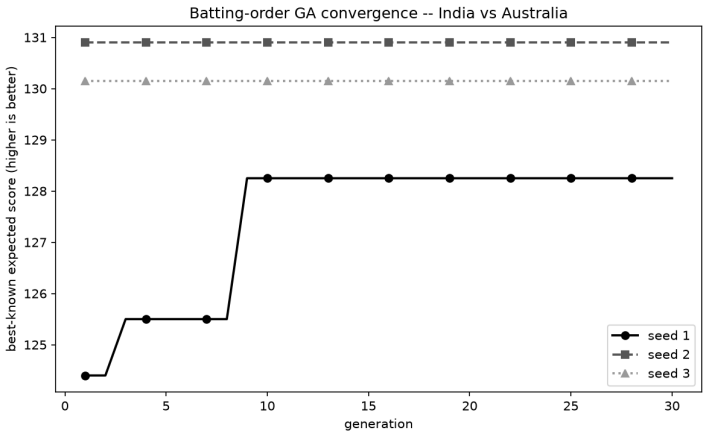


Figure 5.7: Batting-order GA: best-known expected score by generation, three seeds

5.8 Match Simulation and Genetic-Algorithm Optimizer: Training and Evaluation

5.8.1 Match Simulator and Genetic-Algorithm Optimizer Design

A separate, rules-compliant ball-by-ball T20 Monte Carlo match simulator has been built (checked directly against a documented rulebook: over and bowler limits, no-ball/wide handling and free hits, correct strike rotation, and innings-ending conditions), together with two hand-rolled genetic algorithms: one that searches bowler-to-over rotations to minimise expected runs conceded, and one that searches batting order to maximise expected score or chase success probability. On top of this sit win-probability charts with key-moment annotation (milestones, boundaries, hat-tricks, and the largest single-ball win-probability swings in a match).

5.8.2 Genetic-Algorithm Optimizer Results

Both searches converge within the allotted 30 generations and hold their best-known answer once reached, which is the basic behaviour expected of a working genetic algorithm. The two tasks diverge, however, once compared against their respective non-AI baselines. The bowling-rotation GA beat an even-split baseline on every one of the three seeds, by 1.68 runs conceded on average (std 0.95 across seeds – reasonably stable). The batting-order GA did not: against an SR-sorted baseline, one seed finished 1.56 points ahead but the other two finished 7.77 and 2.83 points behind, for a mean 3.0 points below baseline. This is reported as found rather than smoothed over, consistent with this chapter's evaluation discipline. The most likely explanation is that the batting-order search space (permutations of batting groups) is more rugged, or the SR-sorted baseline itself already close to a local optimum, than the population size and generation count (24, 30) used here can reliably escape on a single fixture; this is flagged in Chapter 6 as a concrete tuning item for future work – larger population/generation settings and a larger seed sample – before the batting-order GA can be trusted as an improvement over a simple heuristic.

The recommendation engine, simulator, and dashboard, once wired end-to-end (Section 5.3.3), are reachable through a working web interface rather than existing only as a backend contract; screenshots of that interface are collected in Appendix A.

5.8.3 Evaluation Discussion

Taken together, these results describe a system whose two probabilistic components – the WPA regressors and the batting-order GA – carry real but currently modest signal, while its two combinatorial components – the MILP selector and the bowling-rotation GA – are both reliably correct and measurably better than a naive baseline. This is a coherent, if uneven, picture: the parts of the system that search over a well-defined constraint set (legal XIs, over allocations) are performing exactly as designed, while the parts that must extract a weak statistical signal from single-match outcomes (WPA) or search a more open-ended combinatorial space with a smaller compute budget (batting order) show the honest limits of the current model and hyperparameter choices. None of these limits change the project's core claim: every number in this chapter came from a real, reproducible evaluation run against a real baseline, not an unverified assumption, and the interface built on top of these components is now demonstrably functioning end to end rather than existing only on paper.

5.9 Explainability via SHAP-based Attribution

The composite score was already a fully documented, additive sum of named components (Chapter 2), which gave coarse-grained explainability for free from the start. That still left the learned WPA-regression component of the score as a single aggregate number with no per-feature reason attached to it. SHAP (SHapley Additive exPlanations) values have now been added for both gradient-boosted WPA regressors, so that for any player and fixture the model component of the score can be decomposed into the individual input features that pushed it up or down (recent strike rate against pace, economy at the death, opponent-specific matchup history, and so on), not just its aggregate value. This attribution is surfaced through the same narrative layer that already renders the composite-score breakdown on the dashboard, now extended with a per-player "Why this recommendation" explain view showing the SHAP-attributed feature contributions directly (Appendix A, Figure A.2), so a coach reviewing a borderline pick sees both which score component drove the decision and, within the model component specifically, which underlying feature drove it.

Figure 5.8 ranks the twelve most influential features by mean absolute SHAP value for each of the four WPA regressors (both genders, batting and bowling), computed on a 300-row holdout sample per model using the same SHAP routine the live API calls. Batting WPA is driven mainly by opponent-relative and recent-form contribution features, while bowling WPA is dominated by recent economy rate ahead of the same opponent-strength signal.

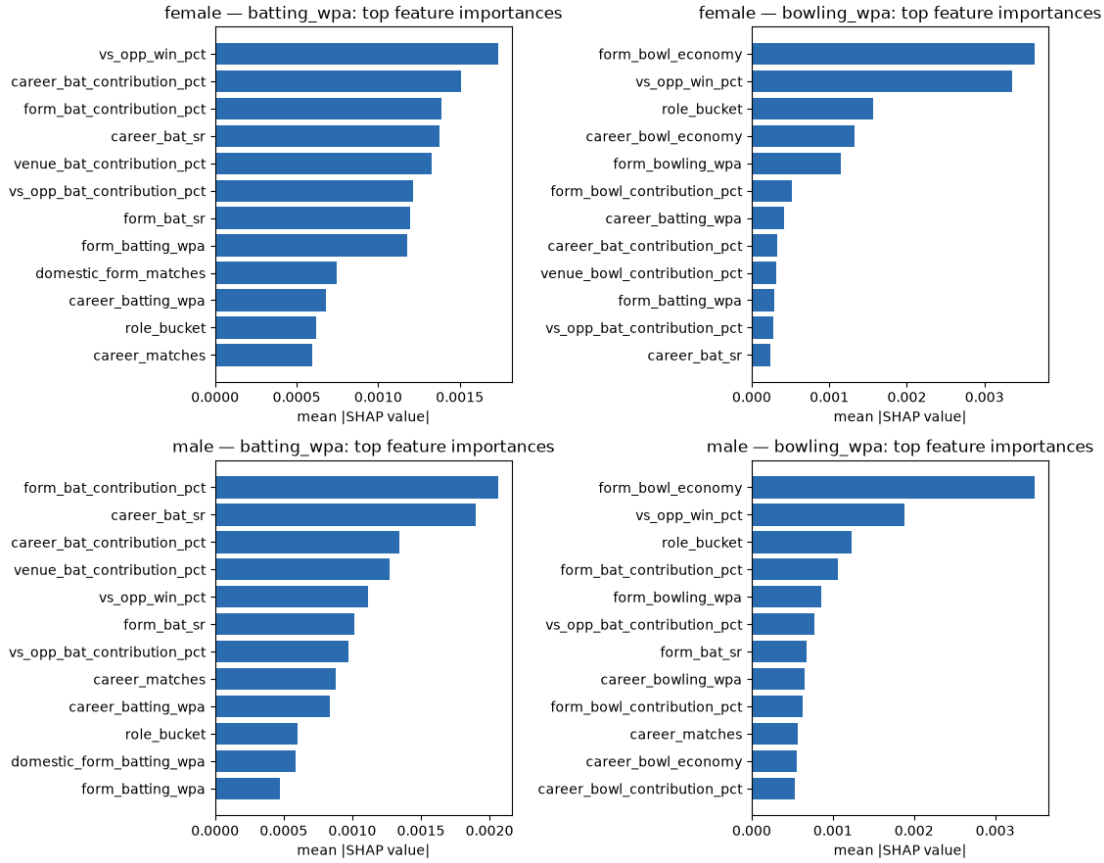


Figure 5.8: Mean absolute SHAP value by feature, top 12 features, both genders and disciplines

Figures 5.9–5.12 show the corresponding beeswarm plots, which add the direction of each feature's effect (colour) to its spread of impact across the holdout sample (position), rather than just its average magnitude.

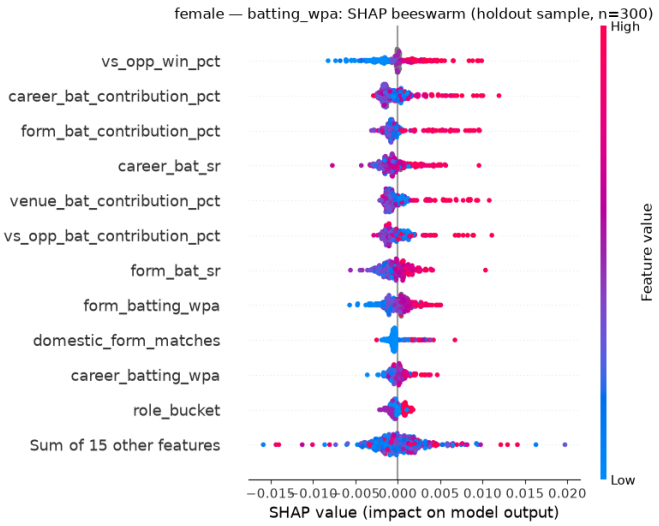


Figure 5.9: SHAP beeswarm, female batting WPA (holdout sample, n = 300)

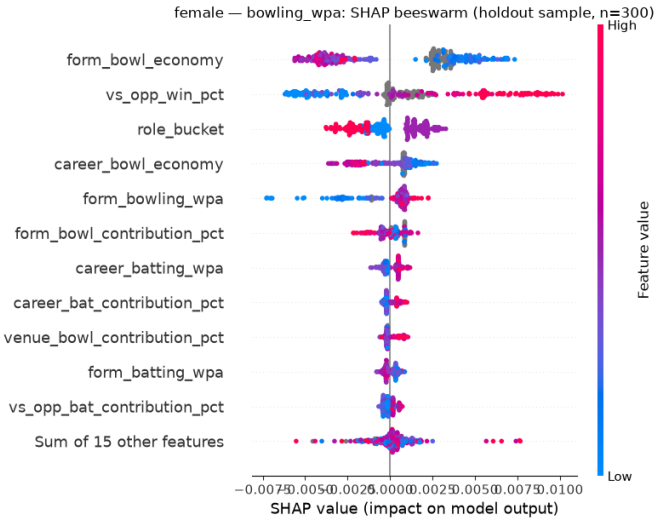


Figure 5.10: SHAP beeswarm, female bowling WPA (holdout sample, n = 300)

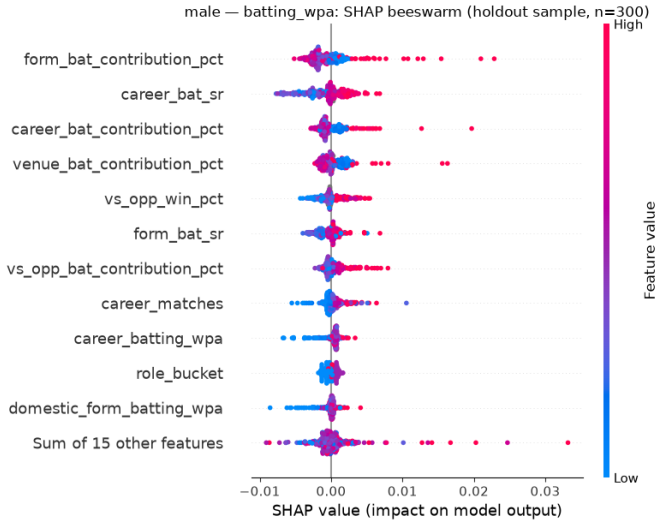


Figure 5.11: SHAP beeswarm, male batting WPA (holdout sample, n = 300)

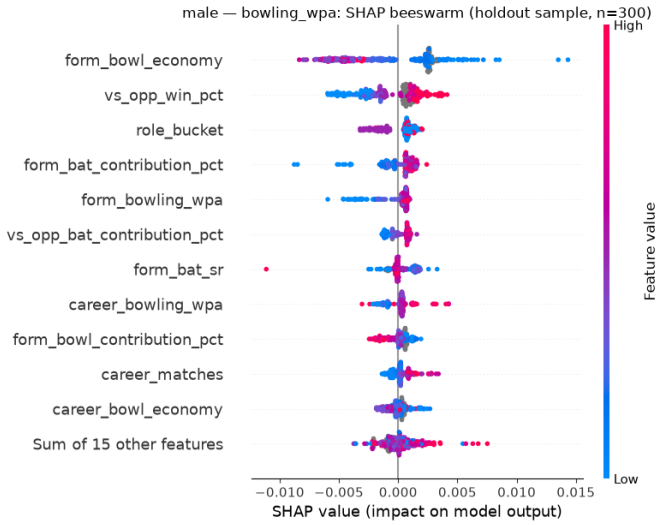


Figure 5.12: SHAP beeswarm, male bowling WPA (holdout sample, n = 300)

5.10 Testing

The data-collection module runs roughly 45 offline unit tests against a schema-accurate pipeline fixture. The model-training module runs a pytest suite of sixteen test files against a small, in-memory SQLite fixture built from scratch in `conftest.py`, covering leakage safety, shrinkage behaviour, eligibility resolution (including the cross-gender collision case), scoring-component semantics, MILP quota satisfaction and fallback behaviour, and team-sheet rendering. Both suites run with no dependency on the live database, so they can be run in a clean environment or continuous integration without any data-collection step first.

CHAPTER 6

CONCLUSION

6.1 Conclusion

Criiq has moved from a proposed architecture to a working, evaluated system. A twelve-stage data pipeline maintains a single, gender-aware database spanning both men's and women's T20 International cricket; two gradient-boosted regressors convert that data into a Win-Probability-Added label for every batting and bowling performance; a seven-component composite score and a mixed-integer linear program turn those labels into a legal, constraint-respecting Playing XI; SHAP-based attribution (Section 5.9) makes the learned component of that score explainable at the level of individual input features; a rules-compliant match simulator and two genetic-algorithm optimizers extend the recommendation into tactical questions of bowling rotation and batting order; and all of this is served end to end through a FastAPI backend and a React dashboard (Appendix A), replacing the placeholder AI module originally used to unblock frontend development. Chapter 5 evaluated each of these components against a real baseline rather than in isolation, and reported what was found honestly: the WPA regressors and the batting-order GA carry a real but modest signal, while the MILP selector and the bowling-rotation GA are both reliably correct and clearly better than a naive alternative. The project's core contribution – a transparent, reproducible, evaluated player-selection pipeline for a player population that existing commercial tooling largely ignores – stands independently of these individual component limits, precisely because every claim in this report is backed by a number a reader can check rather than an assumption asked to be taken on faith.

6.2 Recommendation

Based on the results in Chapter 5, the following are recommended as near-term technical priorities for anyone continuing this work:

- Re-tune the batting-order genetic algorithm before relying on it in any user-facing recommendation: increase population size and generation count beyond the (24, 30) used in this evaluation, and re-run the seed-stability check

across more seeds and more than one fixture before concluding whether the search space itself, rather than just its hyperparameters, is the limiting factor.

- Extend the MILP and overlap validation beyond women's T20I, to men's T20I and to squads with thinner historical coverage, before treating the current 100% constraint-compliance and 68.2% real-XI-overlap results as representative of the whole player population the system is meant to serve.
- Re-run the full evaluation suite on a fixed schedule as new match data accrues, since both the WPA regressors and the win-probability model they depend on are only as current as their last training run.

6.3 Limitation and Future Work

The following limits are documented here rather than glossed over, consistent with the evaluation discipline followed throughout this report:

- Single-match WPA is an inherently high-variance target. The low (0.005–0.014) R^2 reported in Chapter 5 is an honest ceiling for this style of model on this target, not a defect specific to this implementation; a single big innings can dominate the outcome in a way no pre-match feature set can foresee.
- The batting-order genetic algorithm does not yet reliably beat its own baseline. On the one fixture and three seeds tested, its mean result was below a simple SR-sorted heuristic, unlike the bowling-rotation search, which beat its baseline on every seed tested.
- The underlying win-probability model is an empirical, binned estimate with no explicit team-strength or venue-scoring prior, which caps the quality of the WPA label every downstream model and metric depends on.
- Validation coverage is narrower than the system's full scope. The MILP compliance and real-XI-overlap checks, and the genetic-algorithm stability checks, were run on well-covered women's T20I teams and a small number of fixtures and seeds; they have not yet been repeated across men's T20I or thinly-covered squads.
- Style and keeper metadata coverage is partial, and the system currently operates on T20I only; One Day International cricket and men's domestic T20 leagues are scaffolded in the codebase but not enabled.

- The system remains decision-support for a human selector, not a replacement for selectorial judgment; none of its outputs are intended to be acted on without review.

The following extensions were scoped in the original proposal or surfaced during evaluation, but fall outside what this project reached within its timeline:

- Replace the empirical win-probability model with a calibrated one that incorporates team-strength and venue priors, expected to sharpen every downstream WPA-derived metric without changing the recommendation engine's architecture.
- Broaden coverage to One Day International cricket, men's domestic T20 leagues, and full bowling-style and wicket-keeper metadata, so that recommendation quality does not depend on which format, gender, or nation a player represents.
- Add locally hosted, LLM-generated scouting reports, grounded in the same computed numbers the templated explanations and SHAP attributions already use, for longer narrative squad and match-day summaries.
- Pursue a trial deployment with an actual selection panel, particularly at associate and affiliate cricket boards where dedicated analytics infrastructure does not currently exist, to validate the system's practical value beyond the retrospective checks in Chapter 5.

REFERENCES

[1] Cricsheet, "Cricsheet: freely-available structured ball-by-ball data for international and t20 league cricket," https://cricsheet.org/, [Online; accessed 9-Aug-2026].

[2] International Cricket Council, "Icc announces record prize money for women's t20 world cup 2024," ICC News, https://www.icc-cricket.com/tournaments/womens-t20-worldcup/news/icc-announces-record-prize-money-for-women-s-t20-world-cup-2024, 2024, [Online; accessed 9-Aug-2026].

[3] ——, "Record prize money declared for t20 world cup 2024," ICC News, https://www.icc-cricket.com/tournaments/t20cricketworldcup/news/record-prize-money-declared-for-t20-world-cup-2024, 2024, [Online; accessed 9-Aug-2026].

[4] ——, "Icc unveils its new global growth strategy," ICC News, https://www.icc-cricket.com/news/icc-unveils-its-new-global-growth-strategy, 2021, [Online; accessed 9-Aug-2026].

[5] Wikipedia contributors, "Women's twenty20 international," Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Women's_Twenty20_International, [Online; accessed 9-Aug-2026].

[6] P. Puram, S. Roy, D. Srivastav, and A. Gurumurthy, "Understanding the effect of contextual factors and decision making on team performance in twenty20 cricket: an interpretable machine learning approach," *Annals of Operations Research*, vol. 325, no. 1, pp. 261–288, 2023.

[7] S. Chakraborty, A. Mondal, A. Bhattacharjee, A. Mallick, R. Santra, S. Maity, and L. Dey, "Cricket data analytics: Forecasting t20 match winners through machine learning," *International Journal of Knowledge-Based and Intelligent Engineering Systems*, vol. 28, no. 1, pp. 73–92, 2024.

[8] S. Lamsal and D. Kahle, "In-game win prediction models for cricket," in *Recent Advances in Next-Generation Data Science*, ser. Communications in Computer and Information Science, vol. 2158. Springer, 2024, pp. 148–168.

[9] B. Hassan, C. Clough, Y. Siddiqi, R. F. Ali, and M. A. Arshed, "Playerrank: Leveraging learning-to-rank ai for player positioning in cricket," *IEEE Access*, vol. 12, pp. 177 504–177 519, 2024.

[10] I. Wickramasinghe, "Applications of machine learning in cricket: A systematic review," *Machine Learning with Applications*, vol. 10, p. 100435, 2022.

APPENDIX A

User Interface Screenshots

This appendix collects screenshots of the working Criiq interface, referenced from Section 5.3.3 and Chapter 5.

A.1 Dashboard

Figure A.1 shows the coach-facing dashboard, including the composite-score breakdown and form-alert feed.

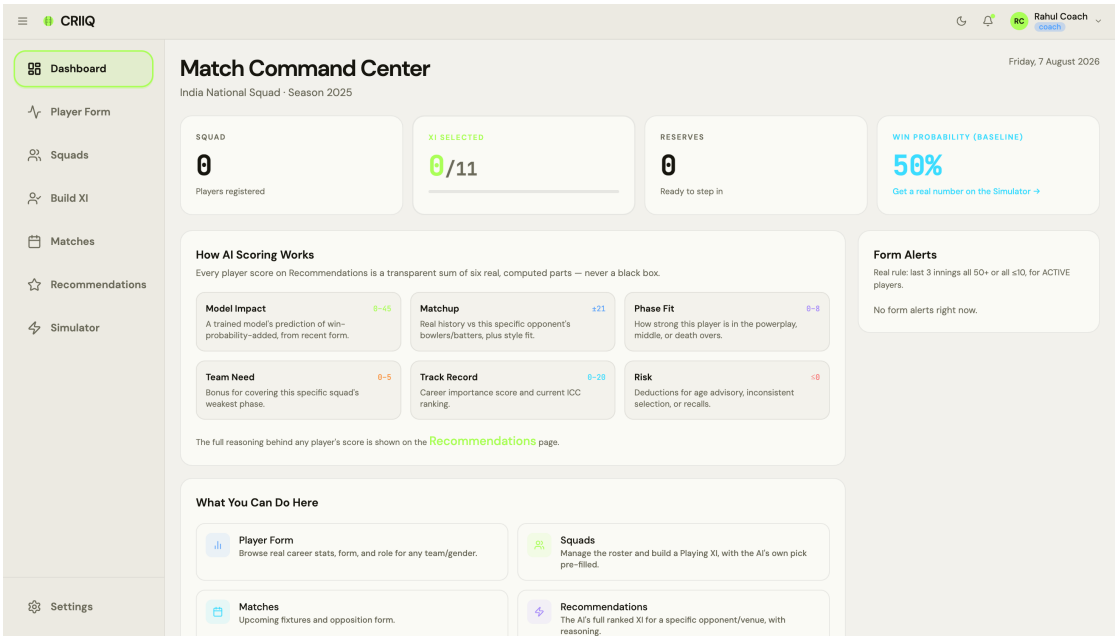


Figure A.1: Match Command Center dashboard

A.2 Recommendation Explainability View

Figure A.2 shows the "Why this recommendation" explain view on the Recommendations page, decomposing a specific player's score into its named components and, per Section 5.9, the SHAP-attributed feature contributions of the model component.

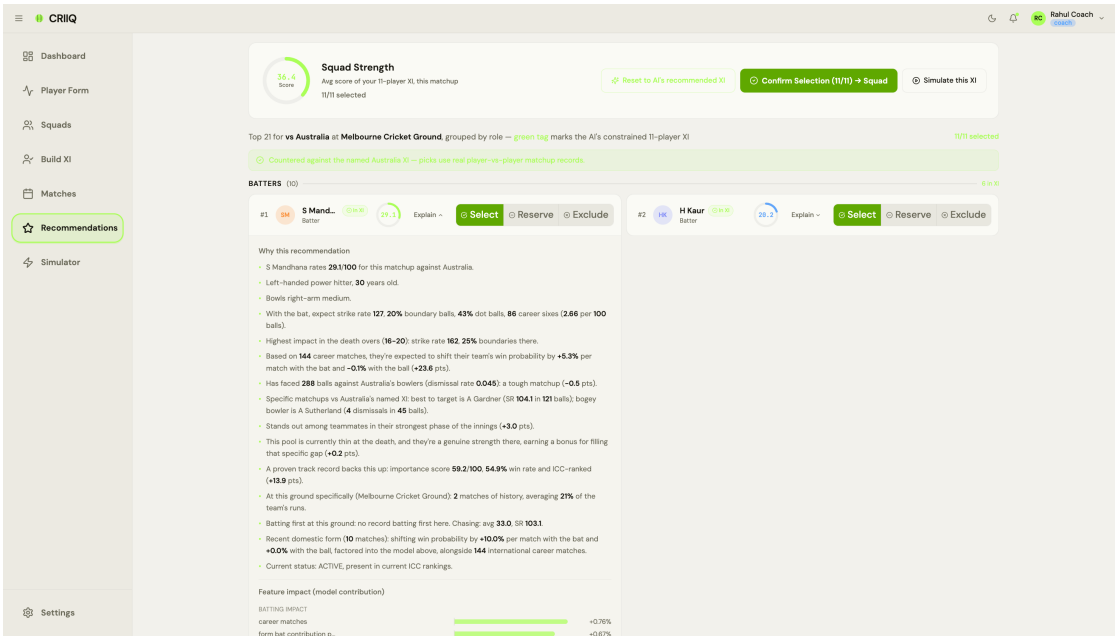


Figure A.2: Recommendations page: "Why this recommendation" explain view with feature-impact attribution

A.3 Match Simulator

Figure A.3 shows the match simulator with live win-probability tracking used to stress-test a chosen XI against an opponent.

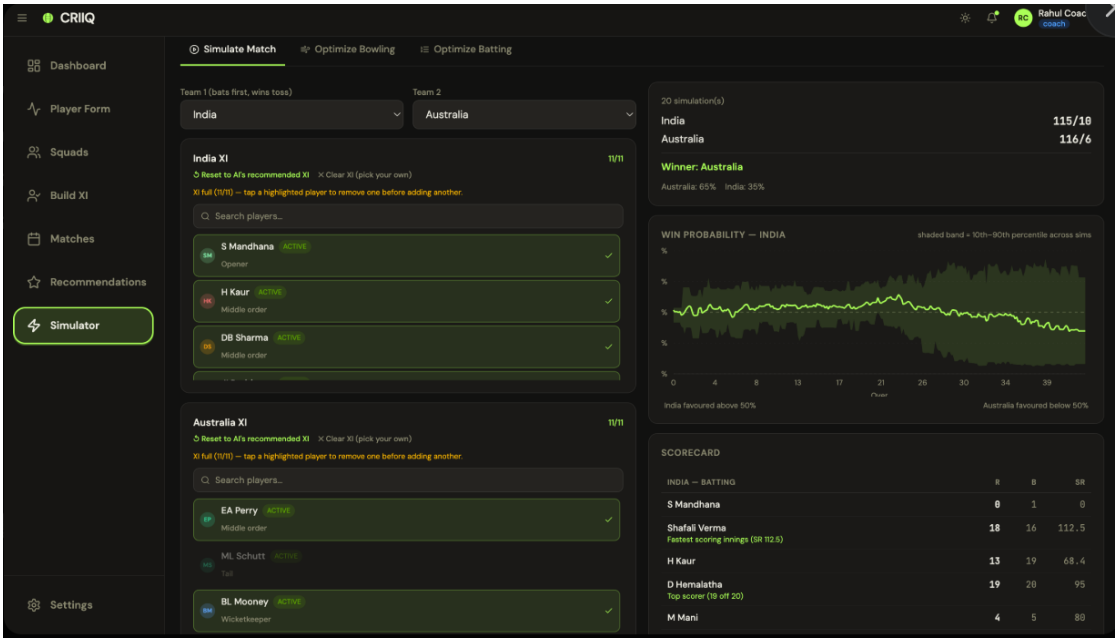


Figure A.3: Match simulator: XI selection, win probability, and ball-by-ball scorecard

A.4 Player Form

Figure A.4 shows the player-form scouting view used to review recent form and head-to-head record before making a selection call.

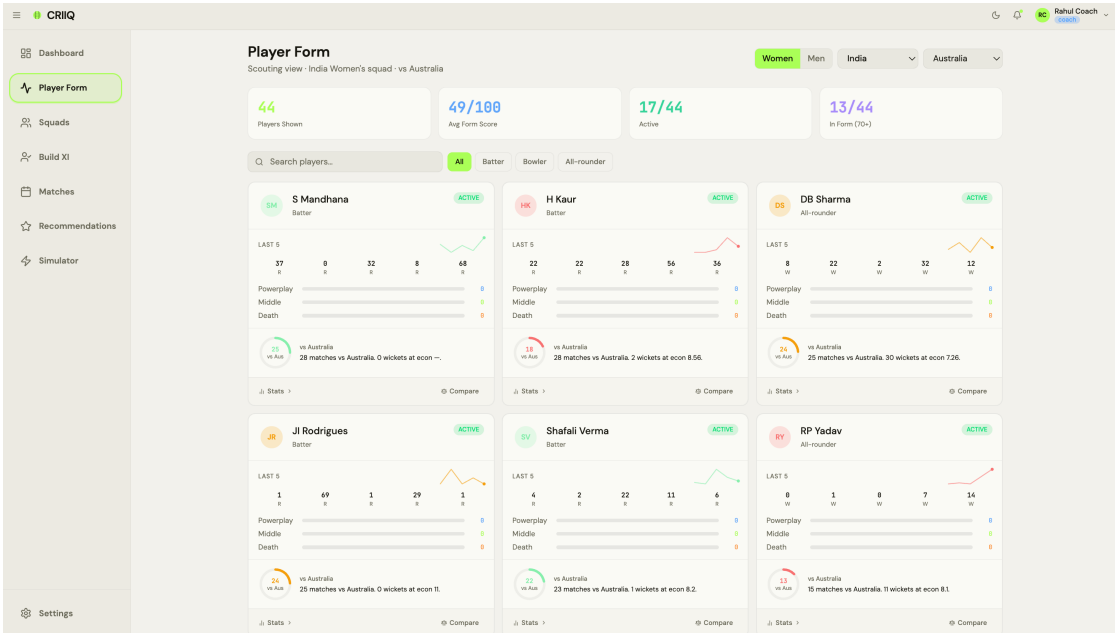


Figure A.4: Player Form scouting view